

Digit Recognition using Artificial Neural Network

RIJAN POKHREL¹, (Student, IOE), and SABIN ACHARYA¹, (Student, IOE)

¹Department of Electronics and Computer Engineering, Thapathali Campus, Thapathali, Nepal (e-mail: rijan.077bei034@tcioe.edu.np)

¹Department of Electronics and Computer Engineering, Thapathali Campus, Thapathali, Nepal (e-mail: sabin.077bei035@tcioe.edu.np)

This lab was performed under the supervision of Er. Dinesh Baniya Kshetri in Department of Electronics and Computer Engineering, Thapathali Campus, IOE, T.U.

• **ABSTRACT** In this lab report, the different configuration of ANNs are explored along with their performances on the modified MNIST dataset. The effect of 3 activation functions; ReLU, Sigmoid and Tanh on model's performance and general tendency of accuracy improvement are explored in detail. The effect of 3 different initialization techniques; Random, Kaiming and Xavier on the model's performance and accuracy improvement are also explored. Also these tendencies are observed for different ANN architecture like having more neurons in single hidden layer vs having 2 hidden layers but less neurons. The results obtained from these different ANN configurations are evaluated based on different performance metric like Accuracy, Train-test Accuracy curves, Confusion Matrix, Heatmap and Classification Report.

• **INDEX TERMS** Artificial Neural Network, Back Propagation, Biases, Epochs, Glorot Initialization, Gradient Descent, Kaiming Initialization, Hyperparameters, Kaiming Initialization, Optimization, ReLU (Rectified Linear Unit), Sigmoid, Tanh, Weights

I. INTRODUCTION

IN Artificial Intelligence, Artificial Neural Networks (ANNs) are models inspired by the structure and function of biological neural networks in the human brain. These complex systems form the backbone of many machine learning and artificial intelligence applications that are designed to recognize complex patterns and solve complex problems that traditional computing finds difficult.

ANNs consist of interconnected nodes or 'neurons' organized into layers. The input layer receives initial data, which then propagates through one or more hidden layers before reaching the output layer. Each connection between neurons carries a weight that determines the strength of the signal passed. As information flows through the network, neurons apply activation functions to its inputs that introduces non-linearity and enables the network to model complex relationships within the data.

ANNs learn from the data through a process called training, where the networks adjust its internal parameters to minimize the difference between

the predictions and the actual outcomes. It is done through the forward propagation of input data, and then backpropagation of errors to fine tune the network's weights. This continuous process continues till the ANN achieves a satisfactory level of accuracy. Despite its strengths, ANNs face certain challenges. It often requires large amounts of training data and computational resources, especially for deep networks with many layers. The "black box" nature of these models makes it difficult to interpret their decision-making processes.

A. HISTORICAL ASPECT OF ANN

ANNs are inspired by the human brain. Scientists tried to develop a new computational model to mimic neural processes. The first milestone for the ANNs was during 50s and 60s when a simple neural network model called perceptron was developed. But due to its limitations (XOR issue: failure of that neural network to learn XOR gate's output), the development of neural network was halted due to the reduced interest and funding that is termed as

AI Winter.

But the ANNs was revived in 1980s where the concept of Backpropagation algorithm that allowed for effective training of multi layer network was introduced. This introduction of the backpropagation proved to be the milestone for the further development of the ANNs. In the further years/decades, the ANNs took a great leap and had a huge achievement due to the increasing computing power, big data availability, and improved Algorithms. Due to all these factors ANNs was breakthrough in various domain performances like image recognition, natural language processing and game development.

B. COMPONENTS OF ANN

1) Neurons(Nodes)

Neurons which are also known as nodes is the fundamental blocks of ANNs. These units are inspired by the biological neurons of the human brain. Basically neurons receive input signals, process them, and produce an output. Each neuron has multiple input connections, each having a weight which determines the importance of that input. The neuron combines these weighted inputs, adds a bias term, and then applies an activation function to produce its output. This activation function introduces non-linearity into the network. Common activation functions are sigmoid, tanh, and ReLU (Rectified Linear Unit). The output of a neuron serves as input to other neurons in subsequent layers that forms the basis of information propagation through the network.

2) Connections(Edges)

Connections in ANNs are the pathways through which information flows between neurons. Each connection has a weight that determines the strength of the signal passing through it. During the training process of an ANN, these weights are adjusted to optimize the network's performance. Positive weights strengthen the signal, while negative weights weaken it. The strength of connections allows the network to learn and represent complex relationships within the data. By modifying connection weights, the network can adapt to recognize patterns and make predictions. The collective arrangement of these weighted connections forms

the architecture of the neural network and plays a crucial role in its ability to process information and learn from data.

3) Layers

Layers in ANNs are organized groups of neurons that process information sequentially. A typical ANN consists of three main types of layers: the input layer, hidden layers, and output layer. The input layer receives the initial data and distributes it to the next layer without performing any computations. Hidden layers, situated between the input and output layers, are where most of the processing occurs. These layers extract and learn features from the input data, with each successive layer potentially learning more abstract features. The output layer produces the final result of the network, that could be a classification, prediction, or any other desired output depending on the task. The number and size of hidden layers can significantly impact the network's capacity to learn and generalize from data that makes layer architecture a crucial aspect of ANN design.

C. TRAINING PROCESS

The primary goal of training is to adjust the network's parameters, mainly the weights and biases, so that it can effectively map inputs to desired outputs with minimal error.

At first the network's weights and biases are set to small random values. Then it is fed to the network in batch. For each input, the network performs a forward pass, where the data flows through the layers that undergoes transformations based on the current weights and activation functions. After the forward pass, the network's output is compared to the actual target output. The difference between the predicted and actual outputs is measured by a loss function, which evaluates how far off the predictions are. This error is then propagated backwards through the network using the backpropagation algorithm, which calculates how each weight contributed to the error.

Based on the calculations, an optimization algorithm, such as gradient descent or its variants, updates the weights and biases to minimize the error. This process is repeated for many epochs

where each epoch is a complete pass through the entire training dataset. Regularization to prevent overfitting and validation using a separate dataset is employed to ensure that the model is learning generalizable patterns rather than memorizing the training data is done in the network.

D. BACKWARD PROPAGATION

It is the process by which the network learns from its errors and adjusts its parameters to improve its performance. Backpropagation works alongside forward propagation to adjust and optimize the network's weights and biases.

The process begins after a forward pass has been completed and the network has produced an output. This output is compared to the true target value, and an error is calculated using a predefined loss function. The main idea of backpropagation is to propagate this error backwards through the network, layer by layer, to determine how each weight contributed to the error. This is achieved by computing the gradient of the loss function with respect to each weight in the network.

II. LITERATURE REVIEW

The paper [1] focuses on the importance of weight initialization in neural networks. The authors argue that while weight initialization may seem less significant compared to other aspects of neural network design, it plays a crucial role in the network's final quality and convergence rate. The paper compares different weight initialization techniques, including zero initialization, random initialization, Xavier initialization, and He-et-al initialization. The authors aim to determine the most effective method for initializing weights to achieve higher accuracy in less time. They emphasize that proper weight initialization is critical for helping neural networks learn good input-output mappings and for improving the chances of finding a good minimum during backpropagation by adequately exploring the search space. Considering this, different weight initialization techniques like Random Initialization using values from Logistic Regression, Kaiming Initialization and Xavier Initialization are deployed in this lab report and their effect in model's performance are assessed.

The paper [2] provides an overview of Artificial Neural Networks (ANNs). It discusses the biological inspiration behind ANNs and their ability to process information in a way similar to the human brain. The authors describe the basic structure of ANNs, including input, hidden, and output layers, and explain how these networks learn through example. The paper covers both supervised and unsupervised training methods, highlighting the process of adjusting weights during training. It also touches on various applications of ANNs, such as pattern recognition, data classification, and adaptive control. The authors emphasize the advantages of ANNs, including adaptive learning, self-organization, real-time operation, and fault tolerance.

Sharma, Sharma, and Athaiya (2020) in [3] provide a comprehensive overview of activation functions in artificial neural networks, emphasizing their critical role in introducing non-linearity, which enables networks to learn complex patterns. Without these functions, neural networks would be limited to behaving like simple linear regression models, significantly reducing their capabilities. The paper discusses various activation functions, including Binary Step, Linear, Sigmoid, Tanh, ReLU, Leaky ReLU, Parametrized ReLU, ELU, Swish, and Softmax, each with its unique advantages and disadvantages. For instance, while ReLU is widely adopted due to its effectiveness, it can lead to "dead neurons," a problem where certain neurons stop learning. In contrast, Sigmoid and Tanh functions, though valuable in specific contexts, are prone to the vanishing gradient problem, which can hinder the training of deep networks. The choice of activation function is shown to depend heavily on the task and context, with Softmax, for example, being commonly used in multi-class classification problems. The authors highlight that no universal rule exists for selecting activation functions, and often, experimentation is necessary to determine the most suitable function for a given problem. Additionally, recent research efforts have focused on developing new activation functions to enhance neural network performance, underscoring the continued importance of this area in the

evolution of deep learning (Sharma et al., 2020). Considering this, different activation functions like ReLU, Sigmoid and Tanh activation functions are employed in this lab report to assess their effect in model's performance.

The paper [4] discusses about the ANN in general, their evolution and their applications. Artificial Neural Networks (ANNs) have emerged as a cornerstone in machine learning and artificial intelligence, drawing inspiration from the human brain's neural structure to emulate complex decision-making processes. The field's origins can be traced to McCulloch and Pitts' 1943 mathematical neuron model, followed by Rosenblatt's influential perceptron model in 1958. ANNs are structured in layers mimicking biological neural networks, with input layers receiving data, hidden layers processing it, and output layers producing results. The development of multilayer perceptrons (MLPs) addressed early limitations in handling non-linear data, marking a significant advancement in ANN capabilities. Training these networks involves adjusting weights and biases through various learning algorithms, including supervised, unsupervised, and reinforcement learning, each finding applications across different domains. ANNs have been widely applied in computer science for tasks such as image recognition and natural language processing, with Convolutional Neural Networks (CNNs) revolutionizing image processing and advancements in NLP leading to improvements in machine translation and sentiment analysis. In the security sector, ANNs contribute to fraud detection and cybersecurity, while in healthcare, they assist in medical image analysis, disease prediction, and personalized patient care. Despite their widespread application and success, challenges persist in optimizing ANN architectures for specific tasks, particularly in training efficiency and managing complex, non-linear data relationships. Ongoing research continues to explore new architectures, training methods, and applications, indicating the growing importance of ANNs across multiple disciplines and their potential for solving complex real-world problems.

III. METHODOLOGY

A. CONCEPTUAL FOUNDATIONS

1) Gradient Descent

It is a basic optimization algorithm used in machine learning to minimize a function, loss function in case of machine learning and deep learning. The basic idea is iteratively adjusting parameters in the opposite direction of the gradient of the function at the present point for finding the function's minima. This process is controlled by an important hyperparameter called learning rate that determines the size of the steps taken in each iteration.

Despite being a powerful optimization algorithm, it faces challenges such as slow convergence for extremely large datasets, sensitive to the learning rate choices and the potential to get stuck in the local minima for non-convex optimization problems. To address these issues, several improved variants have been developed, including Momentum, RMSprop, and Adam, which enhance convergence speed and stability.

The different forms of the Gradient Descent are:

a: Batch Gradient Descent

It uses the entire dataset in each iteration. It can be slow and computationally expensive, especially with large datasets.

b: Stochastic Gradient Descent (SGD)

Instead of using the whole dataset, SGD updates the model parameters using just one random sample at a time. It is faster but can be noisier that leads to the more fluctuations in the optimization paths.

c: Mini-Batch Gradient Descent

It is a compromise between batch and stochastic methods. It updates the model using small random subsets (mini-batches) of data that balances the speed of SGD with the stability of batch gradient descent.

2) Optimizers in Gradient Descent

Optimizers in Gradient Descent are the algorithms that are designed for improving the speed and effectiveness of the optimization process. It is built upon the basic gradient descent algorithm for addressing various challenges and to enhance performance. Optimizers aim to improve convergence speed, handle different types of loss surfaces more

effectively, and solve issues like getting stuck in local minima or saddle points. The choice of optimizer significantly impacts the training process and final performance of machine learning models. Each optimizer has its own strength and is suited to different unique types of problems. Thus the selection of an appropriate optimizer is an important consideration in the model training and development.

Some of the key optimizers are:

a: Momentum

It adds a fraction of the previous update to the current one. It helps to accelerate convergence and reduce oscillations. It's especially helpful in areas where the gradient direction is steady.

b: AdaGrad(Adaptive Gradient)

It adapts the learning rate to the parameters that perform the smaller updates for the frequently occurring features and larger updates the non frequent. It is suited for dealing with sparse data.

c: RMS prop

It solves the AdaGrad's diminishing learning rates by using a moving average of squared gradients. It converges rapidly and is effective for dealing with the non-stationary.

d: Adam(Adaptive Moment Estimation)

It combines ideas from momentum and RMSprop. It computes adaptive learning rates for each parameter and also includes bias correction terms for the first and second moments of the gradients.

3) Learning Rate in Gradient Descent

The learning rate is a critical hyperparameter in gradient descent, an optimization algorithm widely used in training machine learning models. It determines the step size taken in the direction of the negative gradient during each iteration of the algorithm. It controls the speed at which the model's parameters are updated throughout the training process.

The learning rate is crucial because it directly affects how quickly and smoothly the model learns during training. A well-chosen learning rate leads to faster convergence to the optimal solution, while

a poorly chosen one results in slow convergence or even failure to converge at all. If the learning rate is set too high, the algorithm skips the optimal point, that causes it to diverge or oscillate wildly. Conversely, if it's set too low, the algorithm might get stuck in local minima or take an excessively long time to reach the optimal solution. The choice of learning rate depends on the nature of problem and the dataset. Generally the chosen value of learning rate is 0.1.

4) Activation Functions

Activation functions are crucial components in deep learning neural networks. It introduces non-linearity into the network, allowing it to learn complex patterns. It basically transforms the output of a neuron, determining whether and to what extent it should be activated or deactivated. The choice of activation function significantly impacts a neural network's learning capacity, training dynamics, and overall performance.

Some of the most popular activation functions are:

a: Sigmoid Function

It is also known as logistic function. It maps the input values to a range between 0 and 1. Sigmoid has a characteristic S-shaped curve and is particularly useful in the output layer of binary classification problems, where the output is interpreted as a probability. However, it suffers from the vanishing gradient problem for very large or small inputs, which slows down learning in neural networks. The equation of sigmoid function is shown in equation-(3).

b: Hyperbolic Tangent(Tanh)

It is similar to sigmoid function but maps the inputs to a range between -1 and 1. Tanh is zero-centered, which makes it easier for the next layer to learn, as the inputs are centered around zero. Like sigmoid, tanh also suffers from the vanishing gradient problem for extreme inputs. However, its zero-centered nature often makes it preferable to sigmoid in hidden layers. Tanh is commonly used in recurrent neural networks and in situations where negative inputs should be treated differently from positive

ones. The equation of tanh function is shown in equation-(4).

c: Rectified Linear Unit(ReLU)

It is the most popularly used activation function. ReLU has several advantages: it's computationally efficient, helps mitigate the vanishing gradient problem and makes the network sparse by only activating some neurons. However, it also suffer from the "dying ReLU" problem, where neurons can get stuck in a situation where they never activate. The equation of ReLU function is shown in equation-(5).

d: Leaky ReLU

Leaky ReLU is a variation of the standard ReLU designed to address the dying ReLU problem. Instead of mapping all negative inputs to zero, Leaky ReLU allows a small, non-zero gradient when the input is negative. This small slope for negative inputs helps prevent neurons from getting stuck in an inactive state. Leaky ReLU captures most of the benefits of standard ReLU while providing a potential solution to its main drawback. The equation of Leaky ReLU function is shown in equation-(6).

e: SoftMax Activation

It is primarily used in the output layer of multi-class classification problems. It transforms a vector of real numbers into a probability distribution, where the sum of all outputs equals 1. This function focuses the largest values while suppressing those significantly below the maximum. Softmax is particularly useful because it provides a clear probabilistic interpretation of the model's predictions across multiple classes. It's commonly used in combination with cross-entropy loss for training multi-class classification models. The equation of SoftMax function is shown in equation-(7).

5) Weight Initialization in ANN

Weight Initialization is the process of giving the initial values of the weights to the neural network before the training starts. The main goal is to choose the initial values of the weights that facilitates effective learning and avoids issues like vanishing or exploding gradients. It is a crucial aspect of training ANNs. The proper initialization of the

weights before training can significantly impact the speed of convergence and overall performance of the network.

Some of the weight initialization techniques are :

a: Zero Initialization

In this method, all the weights in the neural networks is set to zero at the starting of training. It is generally not recommended. The main issue with this techniques is that it leads to the symmetry problems in the network. When all weights are zero then every neuron in the layer computes the same output and receives the same gradient during backpropagation. This symmetry indicates that the all neurons in the layers will learn the same features that nullifies the benefits of having the multiple neurons. Also in certain activation functions like ReLU, zero initialization leads to dead neurons that never activate and thus never learn.

b: Random Initialization

In this method a random value of the weights are assigned that are drawn from a Gaussian or Uniform distribution. It eliminates the problems of zero initialization that allows the different neurons to learn different features. These values are kept small to prevent the saturation of activations. It can sometime lead to the suboptimal learnings. If the initial values are too large, it lead to exploding gradients where error will grow exponentially during the backpropagation. Conversely if it is too small, it causes vanishing gradients where gradients becomes extremely small, slowing down or halting learning. The effectiveness of random initialization can vary depending on the depth of the network and the choice of activation functions.

c: Xavier/Glorot Initialization

It was developed by Xavier Glorot and Yoshua Bengio. It is a technique especially designed for networks using sigmoid or tanh activation activation functions. In this technique the weights are drawn from the distribution with zero mean and a variance of $2/(n_{in} + n_{out})$, where n_{in} and n_{out} are the number of input and output neurons respectively. The core idea behind Xavier initialization is to maintain the variance of activations and gradients across layers that helps in efficient

backpropagation of gradients. It reduces the problem of exploding or vanishing gradients. By scaling the initial weights based on the number of input and output neurons, Xavier initialization helps ensure that the signal can flow properly through the network from the input to the output, even in deep architectures.

d: Kaiming Initialization

This method was proposed by Kaiming He et al. It is more or less similar to the Xavier Initialization but is specifically designed for networks using ReLU (Rectified Linear Unit) activation functions. It uses a variance of $2/n_{in}$ for the weight distribution, where n_{in} is the number of input neurons. N_{out} is ignored in the variance calculations because ReLU activation functions eliminates out half of the inputs on average, changing the dynamics of information flowing through the network. Kaiming initialization maintains the variance of activations and gradients across layers, specifically designed for ReLU activations. It's especially effective for training deep networks, helping to prevent vanishing gradients and enabling the training of very deep models.

6) Regularization Techniques in ANN

It is the techniques used to prevent overfitting and improve the model's ability to generalize to unseen data. These methods work by adding constraints or modifications to the learning process that discourage the model from fitting too closely to the training data. Regularization helps a model find the right balance: it prevents the model from being too simple (missing important patterns) or too complex (fitting noise). It ensures the model learns what really matters from the data.

Some of the techniques used for regularization are:

a: L2 Regularization(Ridge)

It adds a penalty term to the loss function, proportional to the sum of the squared weights in the network. This encourages the model to use smaller weights distributed across multiple features rather than relying heavily on a few. By penalizing large weights, L2 regularization helps prevent overfitting and improves the model's ability to generalize to

new data. It's particularly effective when dealing with multicollinearity in input features.

b: Dropout

Dropout is a technique where some neurons are randomly turned off during training. This helps prevent the network from relying too much on any single neuron and encourages it to learn more general features. It's like training multiple different versions of the network and then combining their results, which improves the model's performance and robustness.

c: Early Stopping

It monitors the model's performance on a validation set during training. When the validation error starts to increase while the training error continues to decrease, training is halted. This point represents the best trade-off between underfitting and overfitting. Early stopping is simple to implement and effectively prevents the model from overfitting to the training data by learning noise or irregularities specific to the training set.

d: Batch Normalization

It normalizes the inputs of each layer in the network that makes the mean close to 0 and Standard deviation close to 1. This technique addresses the problem of internal covariate shift, where the distribution of each layer's inputs changes during training as the parameters of the previous layers change.

7) Logistic Distribution

This distribution is characterized by a symmetrical, bell-shaped curve that resembles the normal distribution but with heavier tails. The distribution is defined by two key parameters: ' μ ' which represents the mean of the distribution and ' s ' which determines its scale or spread.

The logistic distribution's S-shaped cumulative distribution function also makes it useful for modeling growth processes and in certain economic applications. Its symmetry and heavier tails compared to the normal distribution can make it a more appropriate choice in situations where extreme values are more likely to occur. The values from this distribu-

tion is used to randomly initialize the weights and biases in the ANN of this Lab Report.

The probability density function (PDF) and Cumulative Distribution Function (CDF) of logistic distribution is shown by the equations-(1) and (2) respectively.

B. DATASETS DESCRIPTION

In this lab report, modified MNIST Dataset is used. MNIST dataset is a widely used dataset in the field of machine learning and deep learning for image classification. It was developed by Yann LeCun, Corinna Cortes, and Christopher J.C. Burges in Modified National Institute of Standards and Technology. The used dataset consists of 42000 images of handwritten digits ranging from 0 to 9. Each image has a size of 28x28 pixels, where pixel value ranges from 0 (Black) to 255 (White). The 28 x 28 pixel values are flattened to get the 784 pixel values in the dataset. MNIST is widely used in academic research and plays a significant role in the development of machine learning techniques, particularly in the areas of neural networks and deep learning.

C. SYSTEM BLOCK DIAGRAMS

The system block diagram for the ANN training is shown in the figure-(1). Each block of the block diagram are explained in detail in the following subsections.

D. DATASET ANALYSIS AND VISUALIZATION

In this section, the MNIST dataset was explored and understood. It contained 42000 handwritten digit images. The two sub-steps in this part are explained as follows:

1) Features and Labels Exploration

The features of the dataset is explored and it contained the 784 features containing the pixel value of 28 x 28 pixels image of handwritten digits which were flattened to get the required features value. The labels contained the actual digit represented by the image which were the integers from 0 to 9.

2) Plotting Image Visualization

The plots of some of the individual MNIST digit samples was visualized that allowed for the visual

inspection of handwritten digit characteristics and variations. The flattened array of 784 values are reshaped to 28 x 28 array and then plotted using the Matplotlib library to visualize the digit represented. This process is shown in code snippet-(1).

E. ANN ARCHITECTURE DEVELOPMENT

This part explores how the ANN architecture is designed for the Model Training. The following subsections explore the process in detail.

1) Graphical Visualization of Neural Network using Diagrams

The visual representation of the neural network architecture showing input layer (784 neurons for flattened 28x28 images), hidden layers, and output layer (10 neurons for digit classes) was visualized. For visualization, the diagramming tool 'Draw.io' is used. The weights and biases, their sizes, notations, the activation function used, number of neurons used in each layer are first visually planned using these diagrams. The planned diagrams for different Neural Networks are shown in figures-(2) to (6).

2) Weights Initialization Function

The function to initialize the weights and biases is written. For initialization, three different approaches are used. For most of the cases, the Random values for Logistic Distribution are used. The function that generates these values for the ANN with 1 hidden layer is shown in code snippet-(2). Similarly, one Neural Network each are trained using the values initialized from the Kaiming and Xavier initializations whose code for initialization for the ANN with 1 hidden layer is shown in code snippets-(3) and (4) respectively.

3) Computing Forward Propagation Function

The forward propagation function is implemented that transformed 784 dimensional input layers through layers to produce 10 dimensional output that represents the digit probabilities. The equations-(8) to (11) are used for forward propagation in the ANN with 1 hidden layer and its implementation as function is shown in code snippet-(5). The Relu, Sigmoid and Tanh activation function that are required for forward propagation in differ-

ent ANNs are written as shown in code snippets-(9), (10) and (11) respectively. Also the SoftMax activation function for output layer is written as shown in code snippet-(12).

4) Deriving Backward Propagation Equations

Taking the Binary Cross Entropy as the Loss function, the Backward Propagation equations are derived as per the ANN architecture. The gradients with respect to each of weights and biases are calculated. The backward propagation equations for ANN with 1 hidden layer are shown in equations-(12) to (17) and their derivation process is shown in the appendix. Also the backward propagation equations for ANN with 2 hidden layers are shown in equations-(24) to (32).

5) Computing Backward Propagation Function

The backward propagation function is implemented that computed the contribution matrix for each of the weights and biases matrices that contributed for the loss in the loss function. Using the above mentioned equations, the function was implemented for performing backward propagation. For backward propagation implementation as function in the ANN with 1 hidden layer is shown in code snippet-(6). The derivative of Relu, Sigmoid and Tanh activation function that are required for backward propagation in different ANNs are written as shown in code snippets-(13), (14) and (15) respectively. In backward propagation, for computing the error, the ground truth and prediction should be matrix of same shape. So, to convert the digit value into a prediction type, the custom one hot encoding function is written that converts the labels in the same format of the prediction as shown in the code snippet-(16).

6) Updating Weights Function

This function takes the contribution matrix from the backward propagation and the Weights and biases and subtracts the contribution matrices from their respective weights according to the provided learning rate. Its implementation for ANN with 1 hidden layer is shown in code snippet-(7).

F. MODEL TRAINING

The following steps were performed to train the model on the above mentioned dataset.

1) Label and Feature Separation

The MNIST data is splitted into the pixel intensity (28x28 images flattened to 784-dimensional vectors) and corresponding digit labels (0-9).

2) Train Test Split

The dataset is splitted into the training set and test set for model training. The training set consists of 41000 instances whereas the testing set consists of 1000 instances.

3) Weights Generation and Feed to Model

The weights from initialization function is fed to the model as the initializing weights.

4) Model training on Train Data

The model was then trained on the training data. For this, weights are initialized, then the forward propagation is performed. Then the backward propagation is performed to calculate the contribution matrix and then the weights are updated to reduce the classification error taking the contribution matrix from the backward propagation. The forward propagation, backward propagation and updating weights and biases are performed till a given number of iterations. For this lab report, 1000 iterations are used.

G. MODEL EVALUATION

Different evaluation metrics like accuracy, classification report, confusion matrix and heatmap are used to evaluate the model based on its prediction on the test dataset.

1) Class Prediction with Test Data

The final prediction of the ANN is a list of probability values. When the m test instances are fed to the model for prediction, the model outputs a matrix of m x 10 where there are 10 probability values associated with each instances. The 10 probabilities value correspond with the probability of that instance being the 10 digits from 0-9. The one with highest probability suggests the actual predicted digit. The function that calculates the digit based

on the probability output is written as shown in code snippet-(8). After this decoding, the actual predicted class labels are obtained.

2) Evaluation By Confusion Matrix and Heatmap

The confusion matrix is created using the predicted labels and the true labels and then the confusion matrix is converted to a heatmap for easy visualization and evaluation.

3) Evaluation By Accuracy and Classification Report

The accuracy throughout the iteration is computed using the self defined function that converted the output probabilities into class labels and then compute the accuracy. The function defined for that purpose is shown in code snippet-(17). The overall classification report for the prediction is generated using the scikit learn library.

IV. RESULTS

The results obtained on applying the above described methodology are as follows:

The train and test accuracy curves for the ANN with 1 hidden layer with Relu, Sigmoid, Tanh activation functions and with random, Kaiming, Xavier initialization and for the ANN with 2 hidden layers are shown in figures-(21) to (27) and their respective confusion matrix when their maximum accuracy was taken is shown in figures-(14) to (20). The accuracy curves shows the accuracies of the model on train and test dataset on every 5 iterations for 1000 iterations for each model. From the training accuracy curves shown in figure-(28), it can be seen that the curves except the one using Sigmoid activation function and the one with 2 Hidden Layers reached the high accuracies of about 80-90% in very few number of iterations. The one using sigmoid activation function improved its accuracy very slowly and never reached the accuracy of 90%. The one with 2 hidden layers first increased its accuracy and then decreased a bit and then grew very fast to reach the accuracy of 90% and above. Also for each ANN, the test Accuracy very closely followed the training accuracy. Also, upon observing the confusion matrix, it can be clearly seen that every model is getting confused on predicting the digits 4 and 9, and also between 7 and 9. The

confusion matrix of using Sigmoid activation function i.e. figure-(15) shows more confusion between these terms due to the fact that even the highest accuracy of this ANN is very less than the other ANNs.

The number of iterations that each of the neural network took to reach different accuracy levels are shown in the table-(8). The table shows that the ANN with Random Initialization with Relu Activation function and the ANN with Xavier initialization with Relu Activation Function achieved the different accuracy levels very quickly. Also using more neurons on even a single hidden layer help to reach the accuracy level very quickly. Using 100 neurons in a single layer even outperformed the one using 2 hidden layers with 10 neurons in each for 1000 iterations. However, the time taken for training the one with 100 neurons in single hidden layer was longer than any other ANN's training time.

The classification report for each ANNs are shown in tables-(1) to (7). The classification report shows good precision recall and F1-Score for all the classes. However, these values are very high for labels 0, 1 and 2 whereas very less for label 9. This inference is same as that obtained from the confusion matrix.

V. DISCUSSION AND ANALYSIS

From the above results, the analysis of different results obtained are as follows:

A. DISCUSSION ON ACCURACY CURVE OF 1 HIDDEN LAYERED ANN WITH RELU ACTIVATION FUNCTION

The curve (21) shows how well the neural network learned over time. Ideally both training and testing accuracies should increase rapidly in the early iterations and then plateau at a high level with the testing accuracy closely matching with the training accuracy. This ideal case scenario is greatly matching in the resulting curve. From the curve, it can be seen that the model achieves the final accuracy of approximately 90-91% for both the training and testing that is indicating the good generalization. The slight gap between training and testing accuracies where training is slightly outperforming testing indicates the minor overfitting.

This could be caused by factors like: model's complexity might be slightly higher than necessary for the given dataset, the training data might not fully represent the complexity of the testing data or due to non usage of the regularization technique for penalizing the overfitting. Also the plateauing of accuracy after 400 iterations is indicating that the extended training won't yield any significant improvements compared to the computation requirement it demands.

B. DISCUSSION ON ACCURACY CURVE OF 1 HIDDEN LAYERED ANN WITH SIGMOID ACTIVATION FUNCTION

The learning curve is more gradual compared to other activation functions, with accuracy increasing steadily over the 1000 iterations. The final accuracy reached about 85% for both training and testing sets. The close alignment of training and testing accuracies indicates good generalization, but the overall performance is lower than some other methods. The sigmoid function's vanishing gradient problem may be contributing to the slower learning and lower final accuracy in 1000 iterations. Despite these limitations, the model shows consistent improvement without significant overfitting.

C. DISCUSSION ON ACCURACY CURVE OF 1 HIDDEN LAYERED ANN WITH TANH ACTIVATION FUNCTION

The accuracy increases rapidly in the first 200 iterations and continues to improve more gradually thereafter. The final accuracy reached about 91% for both training and testing sets. There's a very small gap between training and testing accuracies, suggesting excellent generalization. The tanh function's zero-centered nature and broader output range likely contribute to this improved performance. The model shows a good balance between learning speed and final accuracy, making tanh an effective choice for this particular task.

D. DISCUSSION ON ACCURACY CURVE OF 1 HIDDEN LAYERED ANN WITH RELU ACTIVATION FUNCTION AND KAIMING INITIALIZATION

The accuracy is starting low but increases rapidly in the first 200 iterations for both training and testing. After about 400 iterations, the curves flatten reaching a final accuracy around 90-91%. The

training and testing accuracies are very close that indicates good generalization. There's a slight gap between them, with training accuracy marginally higher. This suggests minimal overfitting. Overall the Kaiming initialization have provided a good starting point thus allowing the neural network to learn effectively and achieve high accuracy with balanced performance between training and testing data.

E. DISCUSSION ON ACCURACY CURVE OF 1 HIDDEN LAYERED ANN WITH RELU ACTIVATION FUNCTION AND XAVIER INITIALIZATION

Similar to the Kaiming method, it showed a rapid increase in accuracy during the first 200 iterations. But a noticeable gap between training and testing accuracies is seen as the iterations progressed. The final training accuracy reached about 92%, while the testing accuracy plateaus around 90%. This larger gap suggests slightly more overfitting compared to the Kaiming method. The Xavier initialization still allows for good overall performance, but the model seems to have learned the training data more specifically. This indicates that the Xavier initialization might lead to slightly less generalization compared to Kaiming although both methods results in high accuracy.

F. DISCUSSION ON ACCURACY CURVE OF 1 HIDDEN LAYERED ANN WITH 100 NEURONS IN HIDDEN LAYER WITH RELU ACTIVATION FUNCTION

The curve is showing rapid starting improvement with the accuracy reaching 80% within the first 100 iterations. The rate of improvement then slows but continues steadily. Final accuracies reach about 94% for training and 93% for testing. The slight gap between training and testing accuracies suggests minor overfitting. The quick initial learning and high final accuracy indicate that the chosen number of neurons allows for efficient learning of the dataset's features. However, the small overfitting suggests that further fine-tuning of the neuron count or regularization might be beneficial.

G. DISCUSSION ON ACCURACY CURVE OF 2 HIDDEN LAYERED ANN WITH RELU ACTIVATION FUNCTION ON BOTH LAYERS

The resulting plot shows an interesting pattern. It has a plateau early on the 50-100 iteration before starting the rapid improvement. This indicates the network overcoming initial difficulties in propagating information through multiple layers. The final accuracy reaches about 91% for both training and testing sets. The close alignment of these accuracies demonstrates good generalization despite the increased model complexity. The multiple layers appear to have enabled the network to capture more nuanced patterns in the data, leading to high accuracy without significant overfitting. This architecture seems well-suited to the complexity of the task at hand.

H. DISCUSSION ON TESTING ACCURACY CURVES

This plot (29) compares the test accuracy of various neural network configurations over 1000 iterations. The "Diff. Neurons" configuration shows the fastest initial learning and highest overall accuracy, reaching about 90% quickly. General(ReLU), Kaiming, and Xavier initializations perform similarly, achieving around 88-89% accuracy. The Tanh activation function follows closely behind. The model with 2 hidden layers shows a unique learning pattern, starting slow but catching up to the others by 400 iterations. The Sigmoid activation function consistently underperforms, reaching only about 83% accuracy. All models except Sigmoid show good convergence, with accuracy stabilizing after 400-600 iterations.

I. DISCUSSION ON TRAINING ACCURACY CURVES

The training accuracy plot (28) shows trends similar to the test accuracy, with some differences. The "Diff. Neurons" setup leads in performance, quickly reaching and maintaining around 95% accuracy. Xavier and Kaiming initializations perform slightly better in training than in testing, suggesting minor overfitting. The model with 2 hidden layers initially lags in training but eventually catches up with the top performers. The Tanh activation function performs consistently well in both training and

testing. The Sigmoid activation function shows the lowest performance but has a smoother learning curve in training. Overall, training accuracies are slightly higher than test accuracies, which is normal and suggests good generalization.

J. DISCUSSION ON ACCURACY VS NO. OF ITERATIONS FOR DIFFERENT ANNS

The table (8) gives the comparative analysis of various neural network configurations that is illustrating how different techniques can impact the number of iterations required to achieve specific accuracy levels. The data spans over activation functions (ReLU, Sigmoid, Tanh), weight initialization methods (Kaiming, Xavier), and architectural choices (100 neurons, multiple layers).

It is seen that the ReLU activation is consistently outperforming the Sigmoid and Tanh especially at the higher accuracies. It is most possibly due to the non saturating nature of the relu that helps to maintain constant gradient flow. ReLU, with its simple linear behavior for positive inputs, avoids saturation issues altogether. Tanh performs better than Sigmoid but still struggles compared to ReLU, especially at higher accuracies. It is because both Sigmoid and Tanh saturate for extreme values, but Tanh's zero-centered nature gives it a slight edge over Sigmoid.

The vanishing gradient problem is seen in the Sigmoid activation function. As accuracy tends to increase, the required iterations grow exponentially which ultimately fails to converge at 90% (it is indicated by NAN in the table). This occurs because the sigmoid function compresses large input values into the smaller output range that causes gradients to become vanishingly small during the backpropagation in deeper networks. The Tanh activation function also suffers this problem but it is less significant than the Sigmoid one.

Also it can be seen that the both Xavier and Kaiming initialization methods consistently outperform the basic configurations across various accuracy levels. These advanced initialization techniques address the vanishing and exploding gradient problems. By carefully setting initial weights to maintain appropriate signal magnitude throughout the network, it enabled faster convergence and more

stable training. The table shows that networks initialized with Xavier or Kaming methods typically require fewer iterations to reach the same accuracy levels compared to their counterparts, especially at higher accuracies. This improvement is clearly seen when compared to the performance of basic setups that is using the standard activation functions like Sigmoid or Tanh. This clearly suggests the importance of proper initialization techniques too for better training.

It can also be seen that the Kaming initialization generally performs slightly better than Xavier across most accuracy levels. Only at instance, at 90% accuracy, Kaming requires 795 iterations while Xavier needs 485. At lower accuracy levels, Kaiming was reaching the accuracy levels a bit faster than the Xavier, but when the accuracy level increased, the Kaiming was lagged behind and Xavier took the lead to obtain more accuracy in less iterations.

The better performance of the Kaiming initialization is due to its design that addresses the issues faced by the ReLU activation functions. Kaiming initialization takes into account the non-linearity of ReLU that can cause the variance of the outputs to grow with network depth. By scaling the weights based on the number of input connections, Kaiming helps maintain a consistent variance throughout the network, leading to more stable gradients during training. Ideally also Kaiming should show an advantage over Xavier, especially in deeper networks or those heavily relying on ReLU activations as Xavier initialization is originally developed to work well with the Sigmoid and the Tanh activations.

The table also gives insights into architectural choices: the 100 Neurons configuration shows consistent performance, while Multiple Layers require significantly more iterations at higher accuracies, highlighting the challenges of training deeper networks.

The "100 Neurons" configuration shows a relatively consistent performance across accuracy levels, requiring fewer iterations at lower accuracies (5 iterations for 10-30% accuracy) and gradually increasing to 245 iterations at 90% accuracy. This

configuration offers good capacity for learning but avoids the complexities of very deep networks. This architecture shows efficient learning at lower accuracies and manageable scaling as accuracy requirements increase.

In contrast, the "Multiple Layers" configuration exhibits a more increase in iterations as accuracy improves, ranging from 30 iterations at 20% accuracy to 750 iterations at 90% accuracy. It requires significantly more iterations, especially at higher accuracies, highlighting the difficulty of propagating gradients through many layers.

Theoretically, deeper networks (Multiple Layers) should have greater representational power and potentially achieve higher final accuracies. However, they are also more challenging to train due to issues like vanishing/exploding gradients and the increased complexity of the optimization landscape. The "100 Neurons" configuration, while simpler, shows more consistent and often faster convergence. Also these observation might change when the iterations are increased. But for 1000 iterations, the above mentioned results are observed.

Thus, the observed pattern between the multiple layers and 100 neurons suggests that while deeper networks (Multiple Layers) require more training iterations, they might ultimately achieve higher accuracy levels not shown in this table. The challenge lies in balancing the potential for higher accuracy with the increased computational cost and training complexity of deeper architectures.

VI. CONCLUSION

The comparative analysis of various neural network configurations, activation functions, and weight initialization techniques reveals significant insights into its impact on learning efficiency and performance. Among the different activation function tested, the ReLU activation function consistently outperforms others, such as Sigmoid and Tanh, particularly at higher accuracy levels. This advantage is due to ReLU's non-saturating nature, which maintains gradient flow more effectively and accelerates learning. The sigmoid function, with its tendency to saturate and cause vanishing gradients, struggles to achieve high accuracy, as evidenced by its exponential increase in iterations required to

approach 90% accuracy. In contrast, Tanh performs better than Sigmoid but still falls short of ReLU, demonstrating the challenges inherent in activation functions that suffer from saturation issues. These results perfectly match the theoretical estimations.

Weight initialization methods play a crucial role in determining network performance. Both Kaiming and Xavier initializations outperform traditional approaches, addressing common issues like vanishing and exploding gradients. Kaiming initialization works better than Xavier because it is specifically designed to help ReLU activations train more quickly and smoothly. This results in fewer iterations needed to reach equivalent accuracy levels, that illustrates Kaiming's effectiveness in maintaining consistent variance throughout the network. Xavier initialization is also effective but tends to perform slightly less efficient than kaiming particularly in deeper networks or those heavily reliant on ReLU activations.

Architectural choices also influence the number of iterations required and the accuracy. The "100 Neurons" setup shows steady performance, with training time increasing reasonably as accuracy improves. This configuration balances learning ability with simplicity. In contrast, deeper architectures like "Multiple Layers" require significantly more iterations to achieve similar accuracy levels, reflecting the increased challenges of gradient propagation and optimization in deeper networks.

While deeper networks have the potential for higher accuracy it requires greater computational resources and training time. It highlights the need for careful consideration of network depth and architecture.

Hence the choice of the right activation functions, weight initialization methods and neurons highly impacts the training , computational time efficiency and the accuracy from the neural networks. In our training the RELU , kaiaiming and the 100 neurons had outperformed all the other combinations.

APPENDIX A SYSTEM BLOCK DIAGRAMS AND FLOW CHARTS

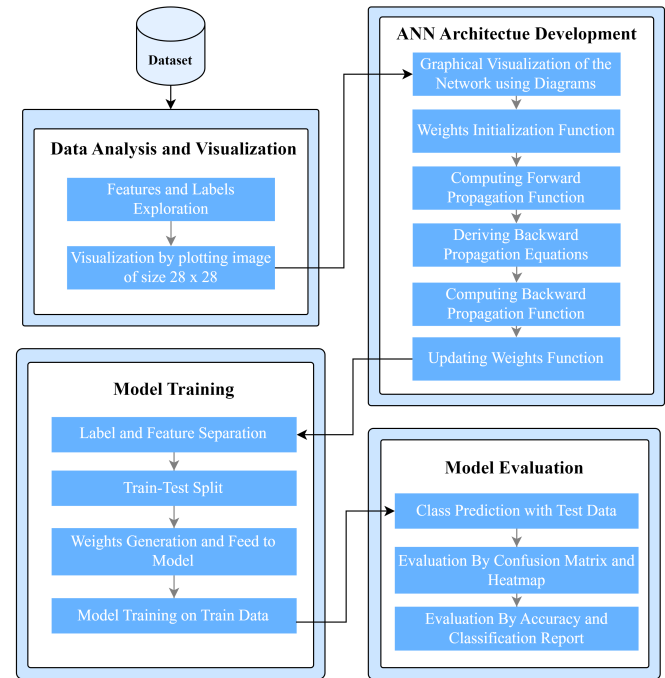


FIGURE 1. System Block Diagram

APPENDIX B ARTIFICIAL NEURAL NETWORK DIAGRAMS

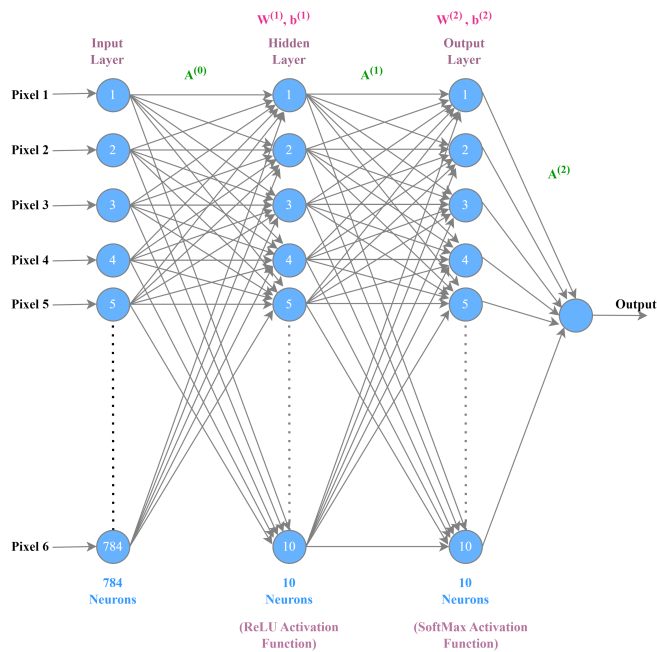


FIGURE 2. Simple ANN with 1 Hidden Layer and ReLU Activation Function

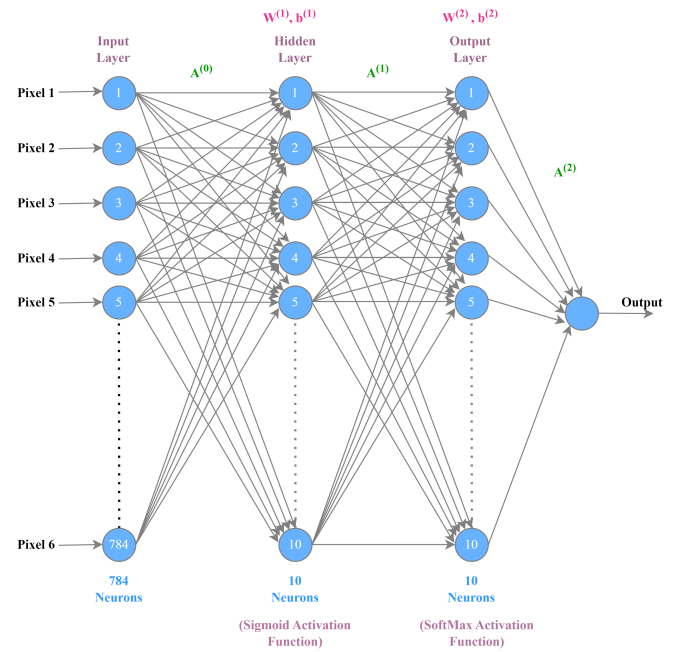


FIGURE 4. Simple ANN with 1 Hidden Layer and Sigmoid Activation Function

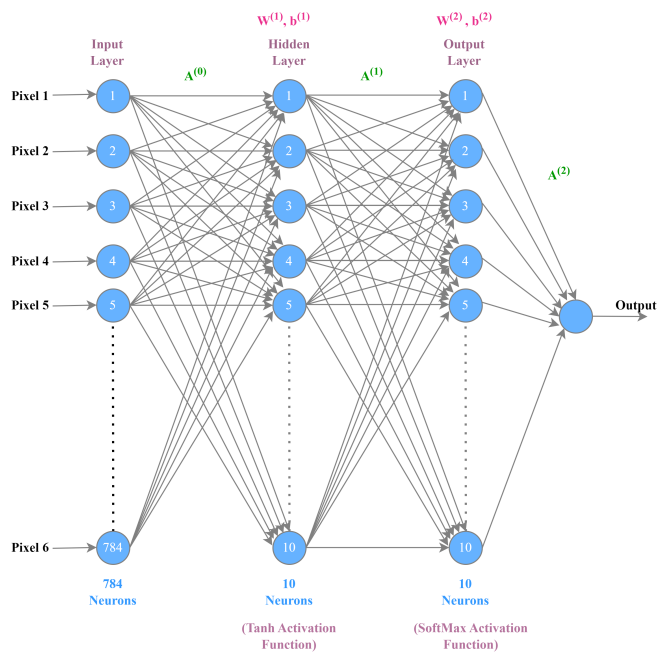


FIGURE 3. Simple ANN with 1 Hidden Layer and Tanh Activation Function

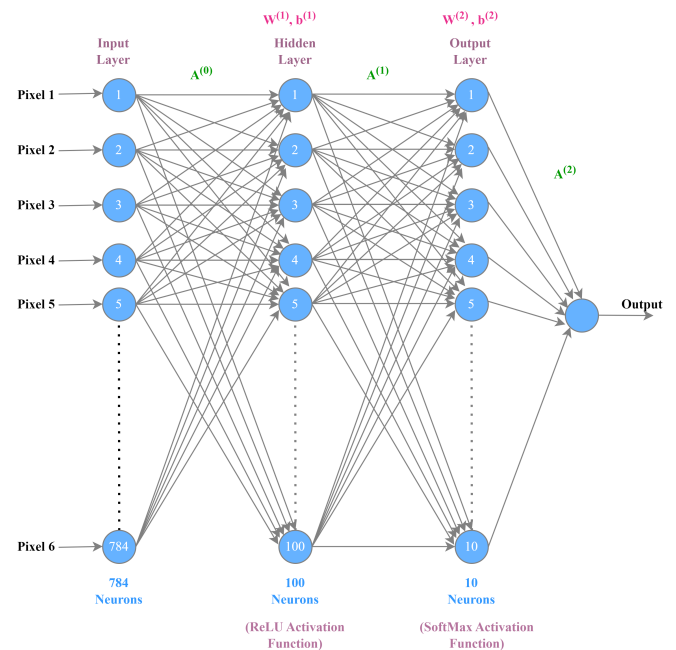


FIGURE 5. Simple ANN with 1 Hidden Layer but 100 neurons in hidden layer

APPENDIX C

ANN WITH 2 HIDDEN LAYERS

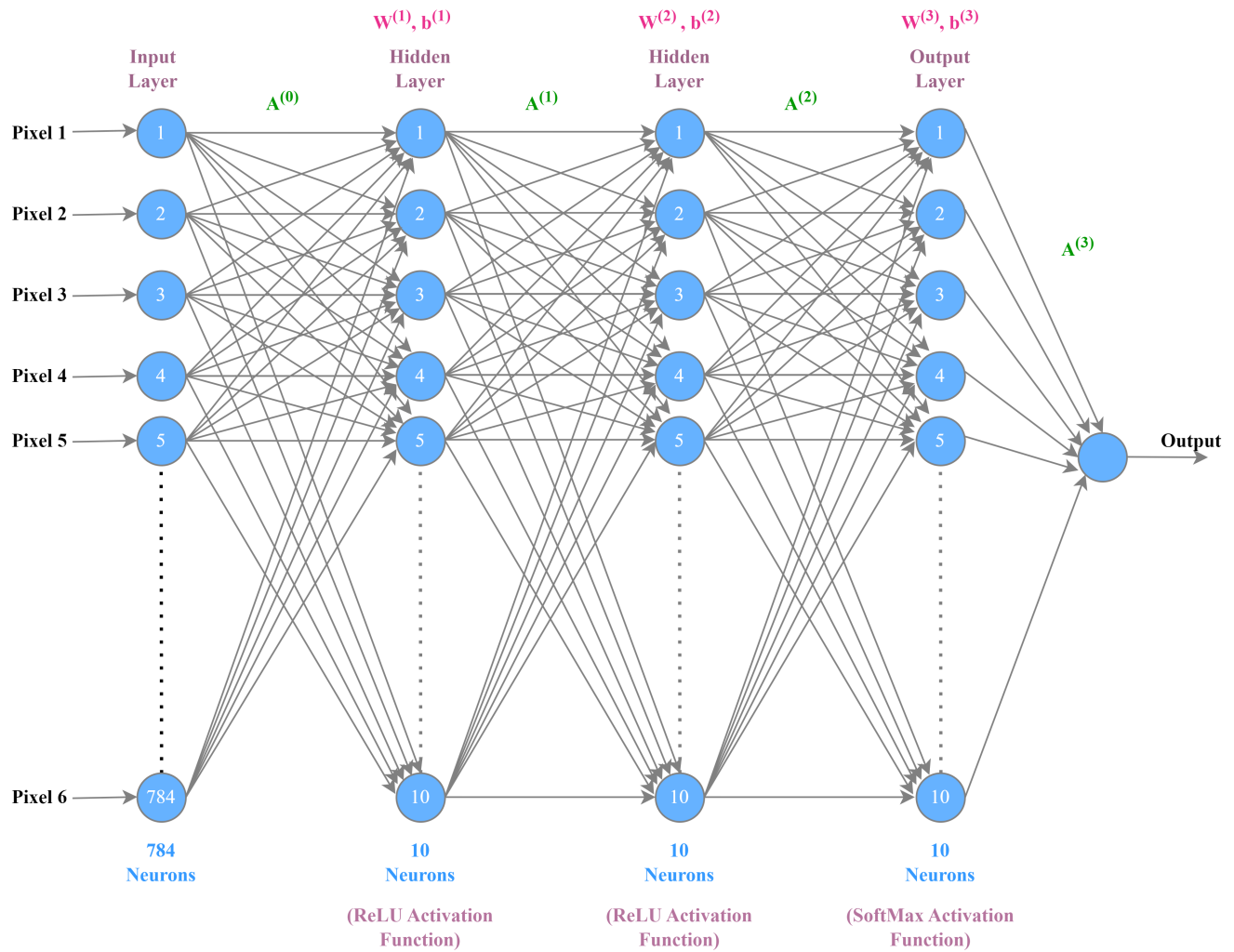


FIGURE 6. Simple ANN with 2 Hidden Layers

APPENDIX D ACTIVATION FUNCTIONS AND THEIR DERIVATIVES

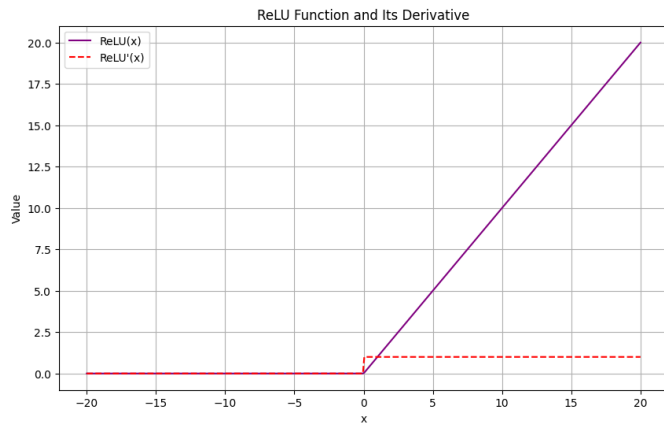


FIGURE 7. ReLU and its Derivative Plot

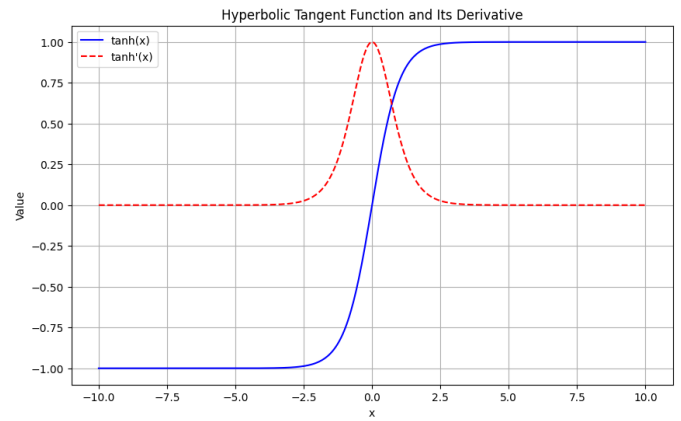


FIGURE 10. Tanh and its Derivative Plot

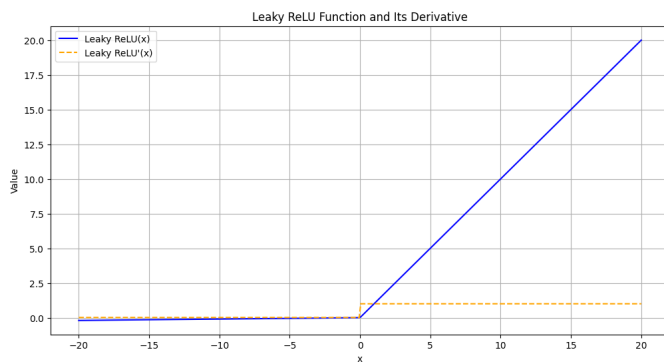


FIGURE 8. Leaky ReLU and its Derivative Plot

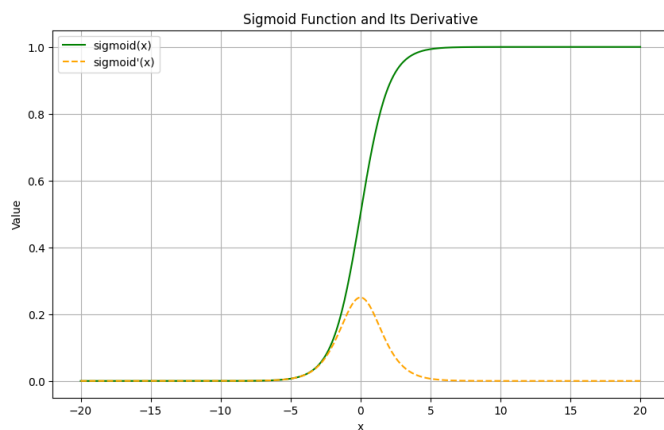


FIGURE 9. Sigmoid and its Derivative Plot

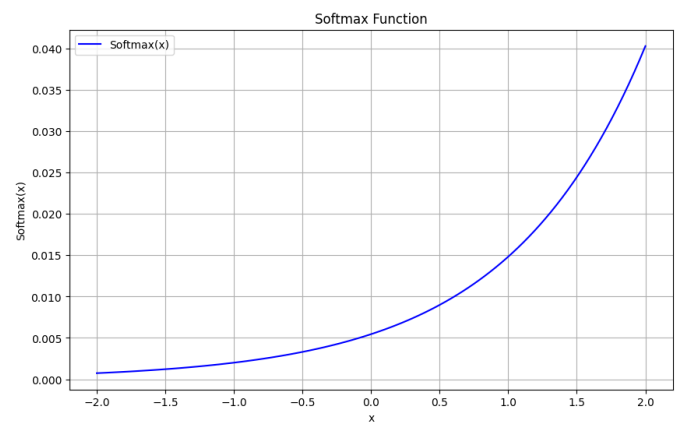


FIGURE 11. SoftMax Plot

APPENDIX E DATASET ANALYSIS AND OUTPUT DIAGRAMS

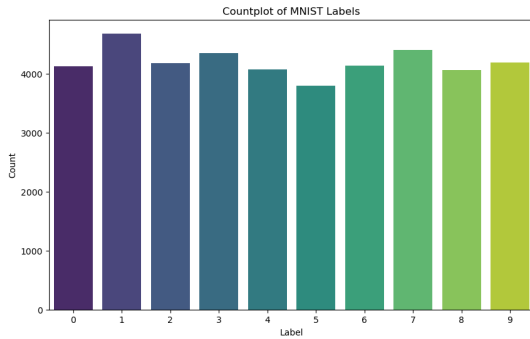


FIGURE 12. Countplot for visualizing count of labels of Voice Dataset

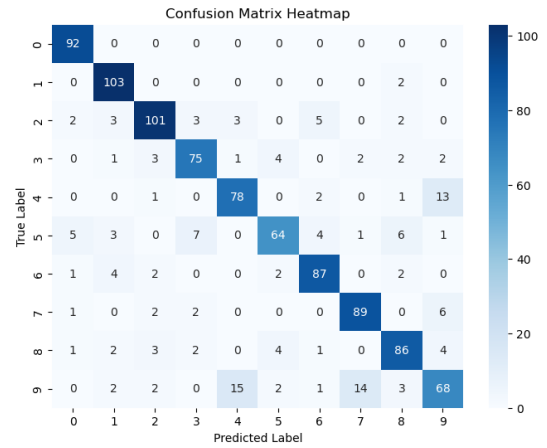


FIGURE 15. Confusion Matrix for 1 Hidden Layer ANN with Sigmoid Activation Function and Random Initialization with Logistic Regression

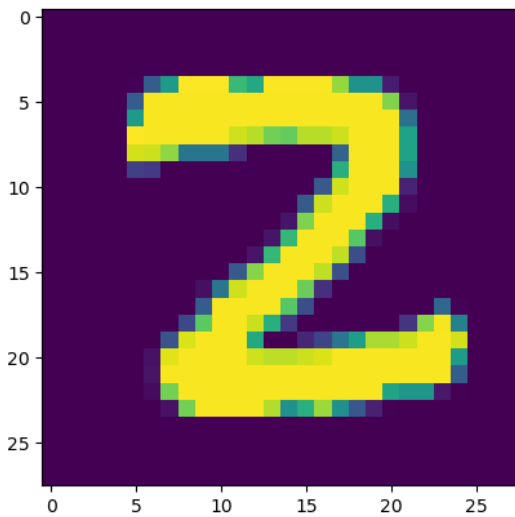


FIGURE 13. Countplot for visualizing count of labels of Voice Dataset

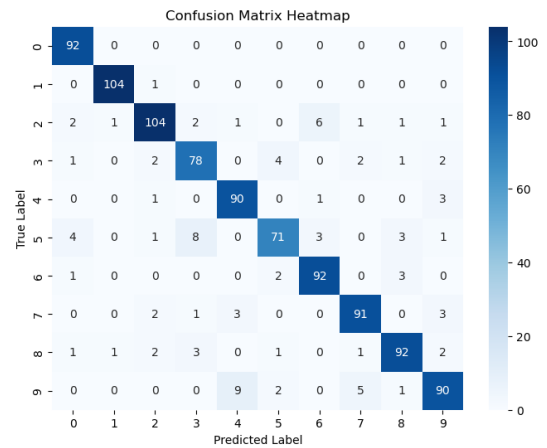


FIGURE 16. Confusion Matrix for 1 Hidden Layer ANN with Tanh Activation Function and Random Initialization with Logistic Regression

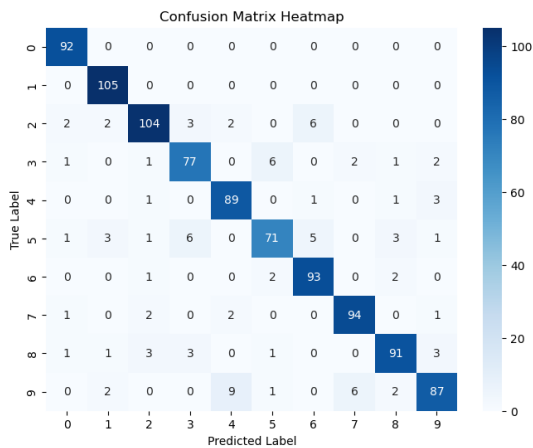


FIGURE 14. Confusion Matrix for 1 Hidden Layer ANN with ReLU Activation Function and Random Initialization with Logistic Regression

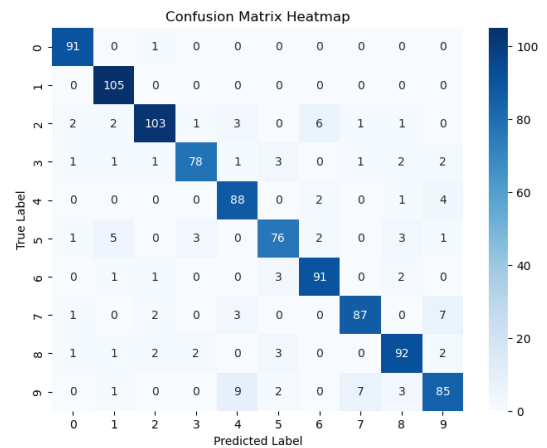


FIGURE 17. Confusion Matrix for 1 Hidden Layer ANN with ReLU Activation Function and Kaiming Initialization

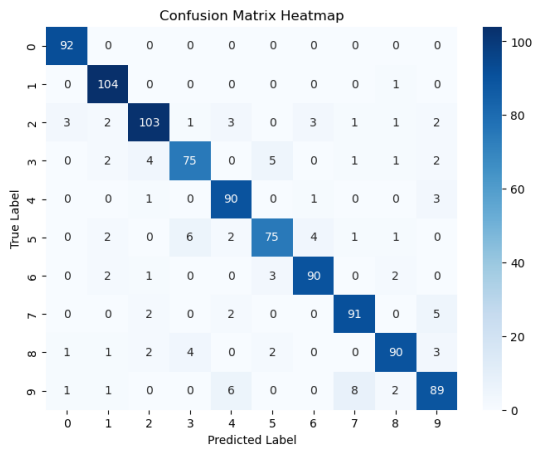


FIGURE 18. Confusion Matrix for 1 Hidden Layer ANN with ReLU Activation Function and Xavier Initialization

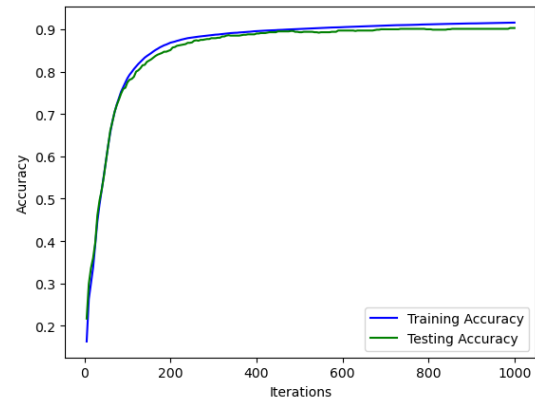


FIGURE 21. Train-Test Accuracy for 1 Hidden Layer ANN with ReLU Activation Function and Random Initialization with Logistic Regression

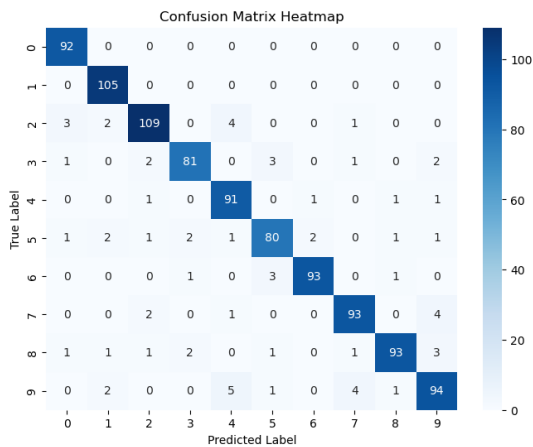


FIGURE 19. Confusion Matrix for 1 Hidden Layer (with 100 neurons instead of 10 neurons) ANN with ReLU Activation Function and Random Initialization with Logistic Regression

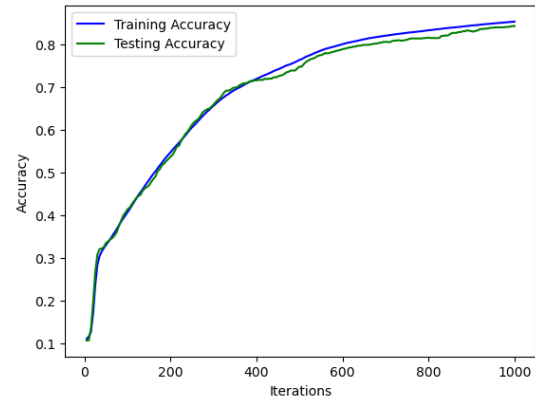


FIGURE 22. Train-Test Accuracy for 1 Hidden Layer ANN with Sigmoid Activation Function and Random Initialization with Logistic Regression

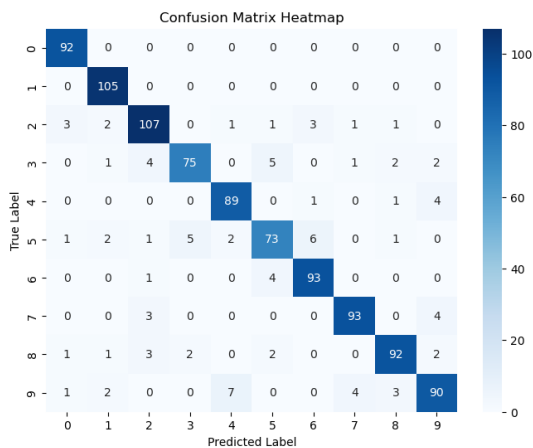


FIGURE 20. Confusion Matrix for 2 Hidden Layer ANN with ReLU Activation Function on Both Layers

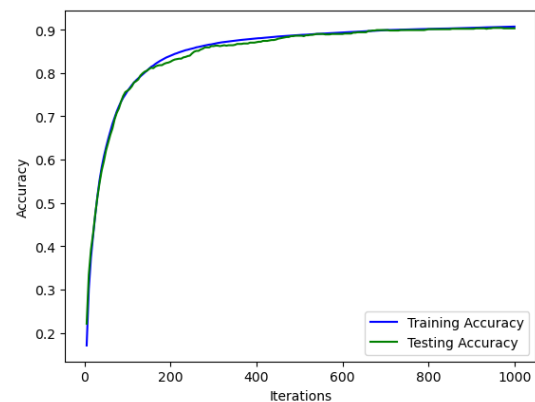


FIGURE 23. Train-Test Accuracy for 1 Hidden Layer ANN with Tanh Activation Function and Random Initialization with Logistic Regression

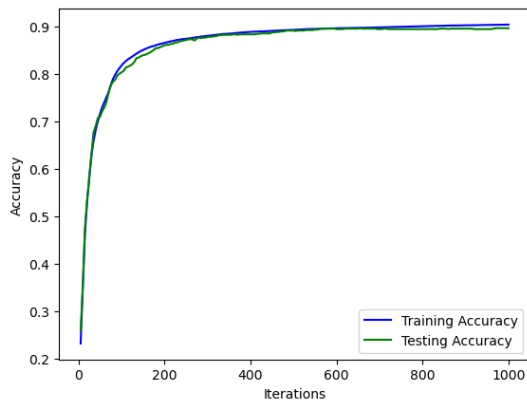


FIGURE 24. Train-Test Accuracy for 1 Hidden Layer ANN with ReLU Activation Function and Kaiming Initialization

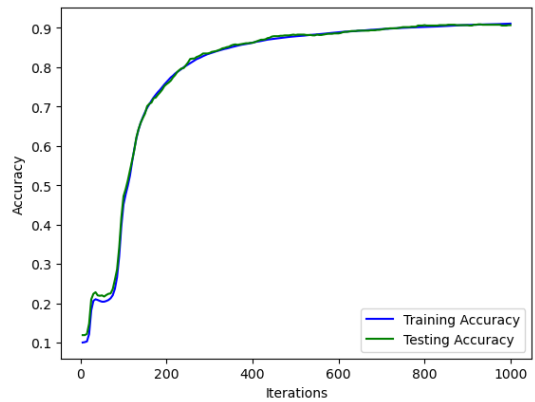


FIGURE 27. Train-Test Accuracy for 2 Hidden Layer ANN with ReLU Activation Function on Both Layers

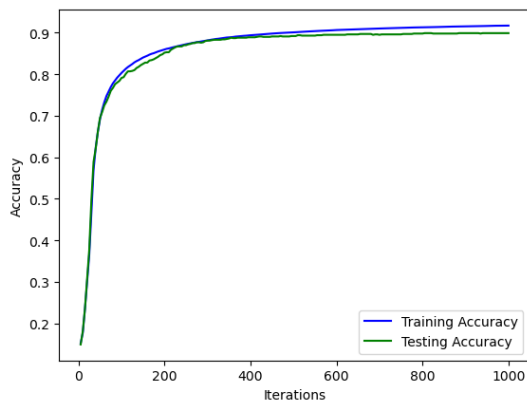


FIGURE 25. Train-Test Accuracy for 1 Hidden Layer ANN with ReLU Activation Function and Xavier Initialization

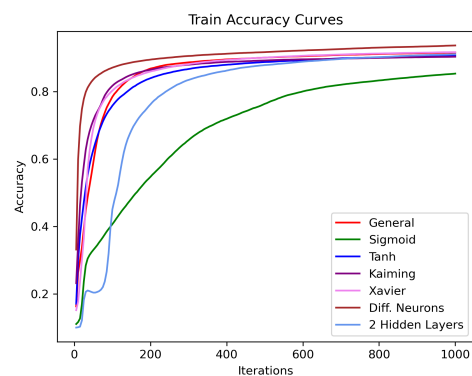


FIGURE 28. Train Accuracy curves for Different ANN

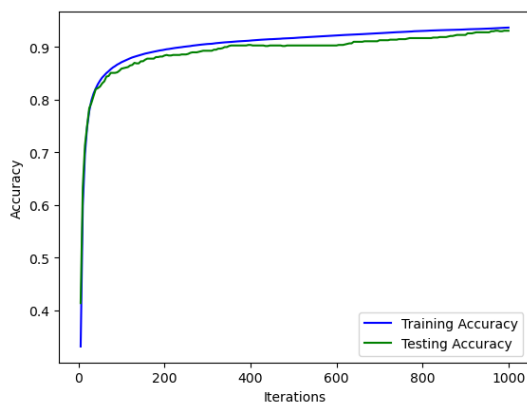


FIGURE 26. Train-Test Accuracy for 1 Hidden Layer (with 100 neurons instead of 10 neurons) ANN with ReLU Activation Function and Random Initialization with Logistic Regression

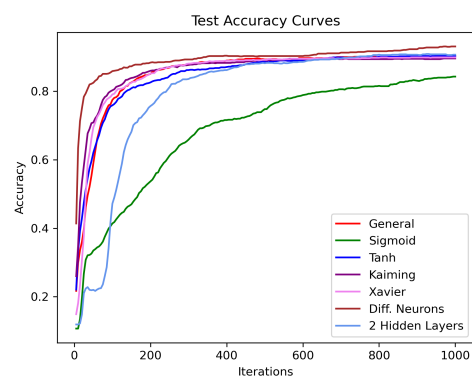


FIGURE 29. Test Accuracy curves for Different ANN

APPENDIX F FORMULA EQUATIONS

A. LOGISTIC DISTRIBUTION EQUATIONS

$$p(x) = \frac{e^{-\left(\frac{x-\mu}{s}\right)}}{s \times \left(1 + e^{-\left(\frac{x-\mu}{s}\right)}\right)^2} \quad (1)$$

$$c(x) = \frac{1}{1 + e^{-\left(\frac{x-\mu}{s}\right)}} \quad (2)$$

where,

s = spread of the distribution,

μ = mean of the distribution.

B. ACTIVATION FUNCTION EQUATIONS

$$\text{Sig}(x) = \frac{1}{1 + e^{-x}} \quad (3)$$

$$\text{Tanh}(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (4)$$

$$\text{ReLU}(x) = \max(0, x) \quad (5)$$

$$\text{LeakyReLU}(x) = \max(0.01 \times x, x) \quad (6)$$

$$\text{SoftMax}(x_i) = \frac{e^{x_i}}{\sum e^{x_j}} \quad (7)$$

where, denomintor represents the sum of exponentiation of all the values in the vector/matrix.

C. FORWARD PROPAGATION EQUATIONS (1 HIDDEN LAYER)

$$Z_{(10xm)}^{(1)} = W_{(10x784)}^{(1)} * A_{(784xm)}^{(0)} + b_{(10x1)}^{(1)} \quad (8)$$

$$A_{(10xm)}^{(1)} = \text{ReLU} \left(Z_{(10xm)}^{(1)} \right) \quad (9)$$

$$Z_{(10xm)}^{(2)} = W_{(10x10)}^{(2)} * A_{(10xm)}^{(1)} + b_{(10x1)}^{(2)} \quad (10)$$

$$A_{(10xm)}^{(2)} = \text{SoftMax} \left(Z_{(10xm)}^{(2)} \right) \quad (11)$$

where,

$W^{(1)}$: Weights between the input layer and the hidden layer,

$W^{(2)}$: Weights between the hidden layer and the output layer,

$A^{(0)}$: Outputs of input layer,

$A^{(1)}$: Outputs of hidden layer,

$A^{(2)}$: Outputs of output layer,

$b^{(1)}$: Biases of the hidden layer,

$b^{(2)}$: Biases of the output layer.

D. BACKWARD PROPAGATION EQUATIONS (1 HIDDEN LAYER)

$$\Delta Z_{(10xm)}^{(2)} = \frac{d(f(y))}{d(Z^{(2)})} = [A_{(10xm)}^{(2)}] - [Y_{(10xm)}] \quad (12)$$

$$\Delta W_{(10x10)}^{(2)} = \frac{d(f(y))}{d(W^{(2)})} = \left(\frac{1}{m} \right) [\Delta Z_{(10xm)}^{(2)}] [A_{(mx10)}^{(1)}]^T \quad (13)$$

$$\Delta b_{(10x1)}^{(2)} = \frac{d(f(y))}{d(b^{(2)})} = \left(\frac{1}{m} \right) \sum [\Delta Z_{(10xm)}^{(2)}] \quad (14)$$

$$\Delta Z_{(10xm)}^{(1)} = \frac{d(f(y))}{d(Z^{(1)})} = [W_{(10x10)}^{(2)}]^T [\Delta Z_{(10xm)}^{(2)}] * [\text{ReLU}'(Z_{(10xm)}^{(1)})] \quad (15)$$

$$\Delta W_{(10x784)}^{(1)} = \frac{d(f(y))}{d(W^{(1)})} = \left(\frac{1}{m} \right) [\Delta Z_{(10xm)}^{(1)}] [A_{(mx784)}^{(0)}]^T \quad (16)$$

$$\Delta b_{(10x1)}^{(1)} = \frac{d(f(y))}{d(b^{(1)})} = \left(\frac{1}{m} \right) \sum [\Delta Z_{(10xm)}^{(1)}] \quad (17)$$

where,

$W^{(1)}$: Weights between the input layer and the hidden layer,

$W^{(2)}$: Weights between the hidden layer and the output layer,

$A^{(0)}$: Outputs of input layer,

$A^{(1)}$: Outputs of hidden layer,

$A^{(2)}$: Outputs of output layer,

$b^{(1)}$: Biases of the hidden layer,

$b^{(2)}$: Biases of the output layer.

$f(y)$: Cost/Loss Function

E. FORWARD PROPAGATION EQUATIONS (2 HIDDEN LAYERS)

$$Z_{(10xm)}^{(1)} = W_{(10x784)}^{(1)} * A_{(784xm)}^{(0)} + b_{(10x1)}^{(1)} \quad (18)$$

$$A_{(10xm)}^{(1)} = \text{ReLU} \left(Z_{(10xm)}^{(1)} \right) \quad (19)$$

$$Z_{(10xm)}^{(2)} = W_{(10x10)}^{(2)} * A_{(10xm)}^{(1)} + b_{(10x1)}^{(2)} \quad (20)$$

$$A_{(10xm)}^{(2)} = \text{ReLU} \left(Z_{(10xm)}^{(2)} \right) \quad (21)$$

$$Z^{(3)}_{(10xm)} = W^{(3)}_{(10x10)} * A^{(2)}_{(10xm)} + b^{(3)}_{(10x1)} \quad (22)$$

$$A^{(3)}_{(10xm)} = SoftMax(Z^{(3)}_{(10xm)}) \quad (23)$$

where,

$W^{(1)}$: Weights between the input layer and the first hidden layer,

$W^{(2)}$: Weights between the first and second hidden layers,

$W^{(3)}$: Weights between the second hidden layer and the output layer,

$A^{(0)}$: Outputs of input layer,

$A^{(1)}$: Outputs of first hidden layer,

$A^{(2)}$: Outputs of second hidden layer,

$A^{(3)}$: Outputs of output layer,

$b^{(1)}$: Biases of the first hidden layer,

$b^{(2)}$: Biases of the second hidden layer,

$b^{(3)}$: Biases of the third hidden layer,

where,

$W^{(1)}$: Weights between the input layer and the first hidden layer,

$W^{(2)}$: Weights between the first and second hidden layers,

$W^{(3)}$: Weights between the second hidden layer and the output layer,

$A^{(0)}$: Outputs of input layer,

$A^{(1)}$: Outputs of first hidden layer,

$A^{(2)}$: Outputs of second hidden layer,

$A^{(3)}$: Outputs of output layer,

$b^{(1)}$: Biases of the first hidden layer,

$b^{(2)}$: Biases of the second hidden layer,

$b^{(3)}$: Biases of the third hidden layer,

$f(y)$: Cost/Loss Function

F. BACKWARD PROPAGATION EQUATIONS (2 HIDDEN LAYERS)

$$\Delta Z^{(3)}_{(10xm)} = \frac{d(f(y))}{d(Z^{(3)})} = [A^{(3)}]_{(10xm)} - [Y]_{(10xm)} \quad (24)$$

$$\Delta W^{(3)}_{(10x10)} = \frac{d(f(y))}{d(W^{(3)})} = \left(\frac{1}{m}\right) [\Delta Z^{(3)}]_{(10xm)} [A^{(2)}]_{(mx10)}^T \quad (25)$$

$$\Delta b^{(3)}_{(10x1)} = \frac{d(f(y))}{d(b^{(3)})} = \left(\frac{1}{m}\right) \sum_{(10x1)} [\Delta Z^{(3)}] \quad (26)$$

$$\Delta Z^{(2)}_{(10xm)} = \frac{d(f(y))}{d(Z^{(2)})} = [W^{(3)}]_{(10x10)}^T [\Delta Z^{(3)}]_{(10xm)} * [ReLU'(Z^{(2)})]_{(10xm)} \quad (27)$$

$$\Delta W^{(2)}_{(10x10)} = \frac{d(f(y))}{d(W^{(2)})} = \left(\frac{1}{m}\right) [\Delta Z^{(2)}]_{(10xm)} [A^{(1)}]_{(mx10)}^T \quad (28)$$

$$\Delta b^{(2)}_{(10x1)} = \frac{d(f(y))}{d(b^{(2)})} = \left(\frac{1}{m}\right) \sum_{(10x1)} [\Delta Z^{(2)}] \quad (29)$$

$$\Delta Z^{(1)}_{(10xm)} = \frac{d(f(y))}{d(Z^{(1)})} = [W^{(2)}]_{(10x10)}^T [\Delta Z^{(2)}]_{(10xm)} * [ReLU'(Z^{(1)})]_{(10xm)} \quad (30)$$

$$\Delta W^{(1)}_{(10x784)} = \frac{d(f(y))}{d(W^{(1)})} = \left(\frac{1}{m}\right) [\Delta Z^{(1)}]_{(10xm)} [A^{(0)}]_{(mx784)}^T \quad (31)$$

$$\Delta b^{(1)}_{(10x1)} = \frac{d(f(y))}{d(b^{(1)})} = \left(\frac{1}{m}\right) \sum_{(10x1)} [\Delta Z^{(1)}] \quad (32)$$

APPENDIX G FORMULA DERIVATIONS

A. DERIVATIVE OF TANH FUNCTION

$$y = \tanh x = \frac{\sinh x}{\cosh x}$$

$$y = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$\begin{aligned} \frac{dy}{dx} &= \frac{(e^x + e^{-x})(e^x + e^{-x}) - (e^x - e^{-x})(e^x - e^{-x})}{(e^x + e^{-x})^2} \\ &= \frac{(e^x + e^{-x})^2 - (e^x - e^{-x})^2}{(e^x + e^{-x})^2} \\ &= \frac{(e^x + e^{-x} + e^x - e^{-x})(e^x + e^{-x} - e^x + e^{-x})}{(e^x + e^{-x})^2} \\ &= \frac{4e^x e^{-x}}{(e^x + e^{-x})^2} \\ &= \frac{4}{(e^x + e^{-x})^2} \\ &= \left(\frac{2}{e^x + e^{-x}} \right)^2 \\ &= \text{sech}^2 x \\ &= 1 - \tanh^2 x \end{aligned}$$

B. DERIVATIVE OF SIGMOID FUNCTION

$$y = \sigma(x) = \frac{1}{1 + e^{-x}}$$

$$\begin{aligned} \frac{dy}{dx} &= \frac{d}{dx} \left(\frac{1}{1 + e^{-x}} \right) \\ &= (-1)(1 + e^{-x})^{-2} \times (-e^{-x}) \\ &= \frac{e^{-x}}{(1 + e^{-x})^2} \\ &= \frac{1}{1 + e^{-x}} \times \frac{e^{-x}}{1 + e^{-x}} \\ &= \frac{1}{1 + e^{-x}} \times \left[\frac{1 + e^{-x}}{1 + e^{-x}} - \frac{1}{1 + e^{-x}} \right] \end{aligned}$$

$$\frac{dy}{dx} = \sigma(x) \times [1 - \sigma(x)]$$

C. DERIVATION OF BACKWARD PROPAGATION EQUATIONS FOR THE NERUAL NETWORK WITH 1 HIDDEN LAYER

1) Derivative of SoftMax

The softmax function can be written in equation form as:

$$f(z_i) = \frac{e^{z_i}}{\sum_{k=1}^n e^{z_k}}$$

where, n = number of elements in the vector,

We have two cases:

1. For $i \neq j$:

$$\begin{aligned} \frac{\partial A_i}{\partial z_j} &= \frac{\partial}{\partial z_j} \left(\frac{e^{z_i}}{\sum_{k=1}^n e^{z_k}} \right) \\ &= \frac{\partial}{\partial z_j} \left(\frac{e^{z_i}}{e^{z_j} + \sum_{k \neq j} e^{z_k}} \right) \\ &= e^{z_i} \times (-1) \times \left(e^{z_j} + \sum_{k \neq j} e^{z_k} \right)^{-2} \times e^{z_j} \\ &= \frac{-e^{z_i} \times e^{z_j}}{\left(e^{z_j} + \sum_{k \neq j} e^{z_k} \right)^2} \\ &= \frac{-e^{z_i} \times e^{z_j}}{\left(\sum_{k=1}^n e^{z_k} \right)^2} \\ &= \left(\frac{e^{z_i}}{\sum_{k=1}^n e^{z_k}} \right) \times \left(-\frac{e^{z_j}}{\sum_{k=1}^n e^{z_k}} \right) \\ &= -A_i \times A_j \end{aligned}$$

2. For $i = j$:

$$\begin{aligned} \frac{\partial A_i}{\partial z_i} &= \frac{\partial}{\partial z_i} \left(\frac{e^{z_i}}{\sum_{k=1}^n e^{z_k}} \right) \\ \frac{\partial A_i}{\partial z_i} &= \frac{\partial}{\partial z_i} \left(\frac{e^{z_i}}{e^{z_i} + \sum_{k \neq i} e^{z_k}} \right) \\ &= \frac{\left(e^{z_i} + \sum_{k \neq i} e^{z_k} \right) \times \frac{\partial}{\partial z_i} (e^{z_i}) - e^{z_i} \times \frac{\partial}{\partial z_i} \left(e^{z_i} + \sum_{k \neq i} e^{z_k} \right)}{\left(e^{z_i} + \sum_{k \neq i} e^{z_k} \right)^2} \end{aligned}$$

$$\begin{aligned}
 &= \frac{(e^{z_i} + \sum_{k \neq i} e^{z_k}) \times (e^{z_i}) - e^{z_i} \times (e^{z_i} + 0)}{(e^{z_i} + \sum_{k \neq i} e^{z_k})^2} \\
 &= \frac{e^{z_i} \times e^{z_i} + \sum_{k \neq i} e^{z_k} \times e^{z_i} - e^{z_i} \times e^{z_i}}{(e^{z_i} + \sum_{k \neq i} e^{z_k})^2} \\
 &= \frac{e^{z_i} \times \sum_{k \neq i} e^{z_k}}{(\sum_{k=1}^n e^{z_k})^2} \\
 &= \left(\frac{e^{z_i}}{\sum_{k=1}^n e^{z_k}} \right) \times \left(\frac{\sum_{k \neq i} e^{z_k}}{\sum_{k=1}^n e^{z_k}} \right) \\
 &= A_i \times (1 - A_i)
 \end{aligned}$$

Hence,

$$\frac{\partial A^i}{\partial z^j} = \begin{cases} -A_i \times A_j & \text{if } i \neq j \\ A_i(1 - A_i) & \text{if } i = j \end{cases}$$

2) Derivation of Backward Prop. Equation (12)

1. Considering categorical cross entropy:

$$L = - \left[y_i \log(A_i) + \sum_{k \neq i}^n y_k \log(A_k) \right]$$

Now, differentiate both sides with respect to z_1 :

$$\begin{aligned}
 \frac{\partial L}{\partial z_i} &= - \frac{\partial}{\partial z_i} \left[y_i \log(A_i) + \sum_{k \neq i}^n y_k \log(A_k) \right] \\
 &= - \left[\frac{y_i}{A_i} \times \frac{\partial A_i}{\partial z_i} \right] - \sum_{k \neq i}^n \frac{y_k}{A_k} \frac{\partial A_k}{\partial z_i} \\
 &= - \left[\frac{y_i}{A_i} \times A_i(1 - A_i) \right] - \sum_{k \neq i}^n \frac{y_k}{A_k} \times (-A_k \times A_i) \\
 &= -y_i \times (1 - A_i) + \sum_{k \neq i}^n y_k \times A_i \\
 &= -y_i + A_i \left(y_i + \sum_{k \neq i}^n y_k \right) \\
 &= -y_i + A_i \sum_{k=1}^n y_k
 \end{aligned}$$

$$= -y_i + A_i \times 1$$

Since, $\sum_{k=1}^n y_k$ is the sum of probabilities and equals 1,

$$\Delta Z_2 = \frac{\partial L}{\partial z_i} = A_i - y_i$$

3) Derivation of Backward Prop. Equation (13)

$$\begin{aligned}
 \frac{\partial L}{\partial W_2} &= \frac{\partial L}{\partial Z_2} \cdot \frac{\partial Z_2}{\partial W_2} \\
 &= \Delta Z_2 \cdot \frac{\partial (W_2 A_1 + b_2)}{\partial W_2} \\
 &= \Delta Z_2 \times A_1
 \end{aligned}$$

For m observations,

$$\Delta W_2 = \frac{\partial L}{\partial w_2} = \left(\frac{1}{m} \right) \Delta Z_2 \times [A_1]^T$$

4) Derivation of Backward Prop. Equation (14)

$$\begin{aligned}
 \frac{\partial L}{\partial b_2} &= \frac{\partial L}{\partial Z_2} \cdot \frac{\partial Z_2}{\partial b_2} \\
 &= \Delta Z_2 \cdot \frac{\partial (w_2 A_1 + b_2)}{\partial b_2} \\
 &= \Delta Z_2 \times 1
 \end{aligned}$$

For m observations,

$$\Delta b_2 = \frac{\partial L}{\partial b_2} = \frac{1}{m} \sum \Delta Z_2$$

5) Derivation of Backward Prop. Equation (15)

$$\begin{aligned}
 \frac{\partial L}{\partial z_i} &= \frac{\partial L}{\partial Z_2} \cdot \frac{\partial Z_2}{\partial A_1} \cdot \frac{\partial A_1}{\partial z_i} \\
 &= \Delta Z_2 \cdot \frac{\partial}{\partial A_1} (W_2 A_1 + b_2) \cdot \frac{\partial}{\partial z_i} (\text{ReLU}(Z_1)) \\
 &= \Delta Z_2 \times W_2 \times \text{ReLU}'(Z_1)
 \end{aligned}$$

For m -observation,

$$\Delta Z_1 = \frac{\partial L}{\partial Z_1} = \Delta Z_2 \times (W_2)^T \cdot * \text{ReLU}'(Z_1)$$

6) Derivation of Backward Prop. Equation (16)

$$\begin{aligned}
 \Delta W_1 &= \frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial z_1} \cdot \frac{\partial Z_1}{\partial W_1} \\
 &= \Delta z_1 \times \frac{\partial}{\partial W_1} (W_1 \times A_0 + b_1) \\
 &= \Delta z_1 \times A_0
 \end{aligned}$$

For m-observation,

$$\Delta w_1 = \left(\frac{1}{m} \right) \Delta z_1 \times (A_0)^T$$

7) Derivation of Backward Prop. Equation (17)

$$\begin{aligned}
 \Delta b_1 &= \frac{\partial L}{\partial b_1} = \frac{\partial L}{\partial Z_1} \cdot \frac{\partial Z_1}{\partial b_1} \\
 &= \Delta z_1 \times \frac{\partial}{\partial b_1} (W_1 A_0 + b_1) \\
 &= \Delta z_1 \times 1
 \end{aligned}$$

For m-observation,

$$\frac{\partial L}{\partial b_1} = \left(\frac{1}{m} \right) \sum (\Delta Z_1)$$

APPENDIX H CODE SNIPPETS

```
1 def plotImage(image):
2     image = np.reshape(image, (28, 28))
3     plt.imshow(image)
```

Listing 1. Digit Visualization by Plotting image

```
1 def parametersInitializer():
2     loc = 0 # mean at 0
3     scale = 1 # steepness of the curve.
4     size_w1 = (10 , 784)
5     size_w2 = (10, 10)
6     size_b1 = 10
7     size_b2 = 10
8
9     logistic_samples = np.random.logistic(
10    loc, scale, size_w1[0] * size_w1[1])
11    logistic_samples = logistic_samples/((
12    abs(logistic_samples)).max()* 2) #
13    scaling to make the values in the range
14    of -0.5 to +0.5
15    W1 = logistic_samples.reshape(size_w1)
16
17    logistic_samples = np.random.logistic(
18    loc, scale, size_w2[0] * size_w2[1])
19    logistic_samples = logistic_samples/((
20    abs(logistic_samples)).max()* 2) #
21    scaling to make the values in the range
22    of -0.5 to +0.5
23    W2 = logistic_samples.reshape(size_w2)
24
25    logistic_samples = np.random.logistic(
26    loc, scale, size_b1)
27    b1 = logistic_samples/((abs(
28    logistic_samples)).max()* 2) # scaling to
29    make the values in the range of -0.5 to
30    +0.5
31
32    logistic_samples = np.random.logistic(
33    loc, scale, size_b2)
34    b2 = logistic_samples/((abs(
35    logistic_samples)).max()* 2) # scaling to
36    make the values in the range of -0.5 to
37    +0.5
38
39    return (W1,b1,W2,b2)
```

Listing 2. Logistic Initialization

```
1 import torch
2 import torch.nn.init as init
3 import numpy as np
4
5 def parametersInitializer():
6     W1 = torch.empty(10 , 784)
7     W2 = torch.empty(10, 10)
8     b1 = torch.empty(10, 1)
9     b2 = torch.empty(10, 1)
10
11    init.kaiming_uniform_(W1, mode='fan_in'
12    ,nonlinearity='relu')
13    init.kaiming_uniform_(W2, mode='fan_in'
14    ,nonlinearity='relu')
15    init.kaiming_uniform_(b1, mode='fan_in'
16    ,nonlinearity='relu')
```

```
14    init.kaiming_uniform_(b2, mode='fan_in'
15    ,nonlinearity='relu')
16
17    b1 = np.array(b1)
18    b2 = np.array(b2)
19    b1 = b1.reshape(-1)
20    b2 = b2.reshape(-1)
21    return (W1,b1,W2,b2)
```

Listing 3. Kaiming Initialization

```
1 import torch
2 import torch.nn.init as init
3 import numpy as np
4
5 def parametersInitializer():
6     W1 = torch.empty(10 , 784)
7     W2 = torch.empty(10, 10)
8     b1 = torch.empty(10, 1)
9     b2 = torch.empty(10, 1)
10
11    init.xavier_uniform_(W1, gain = init.
12    calculate_gain('relu'))
13    init.xavier_uniform_(W2, gain = init.
14    calculate_gain('relu'))
15    init.xavier_uniform_(b1, gain = init.
16    calculate_gain('relu'))
17    init.xavier_uniform_(b2, gain = init.
18    calculate_gain('relu'))
19
20    b1 = np.array(b1)
21    b2 = np.array(b2)
22    b1 = b1.reshape(-1)
23    b2 = b2.reshape(-1)
24    return (W1,b1,W2,b2)
```

Listing 4. Xavier Initialization

```
1 def forwardPropagation(W1, b1, W2, b2, A0):
2     Z1 = np.matmul(W1, A0)
3     Z1 = Z1 + np.repeat(b1, Z1.shape[1]).
4     reshape(Z1.shape)
5     A1 = ReLU(Z1)
6     Z2 = np.matmul(W2, A1)
7     Z2 = Z2 + np.repeat(b2, Z2.shape[1]).
8     reshape(Z2.shape)
9     A2 = softmax(Z2)
10    return (Z1, A1, Z2, A2)
```

Listing 5. Forward Propagation Function

```
1 def backwardPropagation(Z1, A1, Z2, A2, W1,
2    W2, A0, Y):
3     Y = oneHotEncoding(Y)
4     m = A0.shape[1]
5     dZ2 = A2 - Y
6     dW2 = (1/m) * np.matmul(dZ2, A1.T)
7     sumDZ2 = [sum(x) for x in dZ2] # summing
8     the elements in the row to get the
9     matrix
10    db2 = (1/m) * np.array(sumDZ2)
11    dZ1 = np.matmul(W2.T, dZ2) *
12    ReLUDerivative(Z1)
13    dW1 = (1/m) * np.matmul(dZ1, A0.T)
14    sumDZ1 = [sum(x) for x in dZ1] # summing
15    the elements in the row to get the
16    matrix
```

```

11 db1 = (1/m) * np.array(sumDZ1)
12 return (dW1, db1, dW2, db2)

```

Listing 6. Backward Propagation Function

```

1 def updateParameters(W1, b1, W2, b2, dW1,
2 db1, dW2, db2, alpha):
3 W1 = W1 - alpha * dW1
4 b1 = b1 - alpha * db1
5 W2 = W2 - alpha * dW2
6 b2 = b2 - alpha * db2
7 return (W1, b1, W2, b2)

```

Listing 7. Update Parameters Function

```

1 def obtainPrediction(out):
2 out = out.T
3 output = np.array([np.where(x == max(x))
4 [0][0] for x in out])
5 return output;
6 # Testing the obtainPrediction function:
7 correct output should be [4, 3]
8 obtainPrediction(np.array([[0.1, 0.004,
9 0.03, 0, 0.7, 0.006, 0.06, 0.1, 0.1,
10 0.1],
11 [0.1, 0.004, 0.03, 0.7, 0, 0.006, 0.06,
12 0.1, 0.1, 0.1]]).T)

```

Listing 8. Obtain Prediction Function

```

1 # ReLU(x) = {0 if x<0
2 # x if x>=0}
3
4 def ReLU(input):
5 input = np.array(input)
6 shape = input.shape
7 input = input.reshape(-1)
8 input = np.array([max(0, x) for x in
9 input])
10 input = input.reshape(shape)
11 return input
12 # Testing the working of the defined ReLU
13 function
14 ReLU([[3, 5, 5], [3, -1, -10]])

```

Listing 9. ReLU Function

```

1 # Sigmoid(x) = 1 / (1 + exp(-x) )
2
3 def Sigmoid(input):
4 input = np.array(input)
5 shape = input.shape
6 input = input.reshape(-1)
7 input = np.array([ 1 / ( 1 + math.exp
8 (-x) ) ) for x in input])
9 input = input.reshape(shape)
10 return input
11 # Testing the working of the defined Sigmoid
12 function
13 Sigmoid([[3, 5, 5], [3, -1, -10]])

```

Listing 10. Sigmoid Function

```

1 # Tanh(x) = (exp(x) - exp(-x)) / (exp(x) +
2 exp(-x))
3 def Tanh(input):
4 input = np.array(input)
5 shape = input.shape
6 input = input.reshape(-1)
7 input = np.array([math.tanh(x) for x in
8 input])
9 input = input.reshape(shape)
10 return input
11 # Testing the working of the defined Tanh
12 function
13 Tanh([[3, 5, 5], [3, -1, -10]])

```

Listing 11. Tanh Function

```

1 import math
2 def softmax(input):
3 vect_list = np.array(input).T
4 for i in range(len(vect_list)):
5 vector = np.array([math.exp(x) for x
6 in vect_list[i]])
7 denominator = sum(vector)
8 vect_list[i] = vector/denominator
9 return vect_list.T

```

Listing 12. SoftMax Function

```

1 # ReLU'(x) = {0 if x<0
2 # 1 if x>=0}
3
4 def ReLUDerivative(input):
5 input = np.array(input)
6 shape = input.shape
7 input = input.reshape(-1)
8 input = np.array([1 if x>=0 else 0 for x
9 in input])
10 input = input.reshape(shape)
11 return input

```

Listing 13. ReLU Derivative Function

```

1 # Sigmoid'(x) = exp(x) / (1 + exp(-x) )^2 =
2 sigmoid * (1 - sigmoid)
3
4 def SigmoidDerivative(input):
5 input = np.array(input)
6 shape = input.shape
7 input = input.reshape(-1)
8 input = np.array([ ( math.exp(-x) / (1
9 + math.exp(-x))^2 ) for x in input])
10 input = input.reshape(shape)
11 return input

```

Listing 14. Sigmoid Derivative Function

```

1 # Tanh'(x) = 1 - Tanh^2(x)
2
3 def TanhDerivative(input):
4 input = np.array(input)
5 shape = input.shape
6 input = input.reshape(-1)
7 input = np.array([ ( 1- math.tanh(x)**2
8 ) for x in input])
9 input = input.reshape(shape)
10 return input

```

Listing 15. Tanh Derivative Function

```
1 def oneHotEncoding(Y):
2     out = np.zeros((len(Y),10), dtype= '
        uint8')
3     for i in range(len(Y)):
4         item = Y[i]
5         out[i, item] = 1
6     return out.T
7
8 # Testing the Function that Performs One Hot
    Encoding
9 pd.DataFrame(oneHotEncoding([0, 3, 4, 5, 1,
        5, 3, 5, 1, 5, 1, 1, 4, 6, 7, 8, 9]))
```

Listing 16. One Hot Encoding Function

```
1 def computeAccuracy(A2, Y):
2     Y = oneHotEncoding(Y)
3     if(np.array(A2).shape != np.array(Y).
        shape):
4         raise Exception('Given Two arrays
        must be of same shape')
5     pred = obtainPrediction(A2)
6     true = obtainPrediction(Y)
7     correct_count = 0;
8     for i in range(len(pred)):
9         if(pred[i] == true[i]):
10             correct_count += 1
11     return correct_count/len(pred);
```

Listing 17. Compute Accuracy Function

TABLE 1. Classification Report For 1 Hidden Layer ANN with ReLU Activation Function and Random Initialization with Logistic Regression

Class	Precision	Recall	F1-Score
0	0.94	1.00	0.97
1	0.93	1.00	0.96
2	0.92	0.87	0.90
3	0.87	0.86	0.86
4	0.87	0.94	0.90
5	0.88	0.78	0.83
6	0.89	0.95	0.92
7	0.92	0.94	0.93
8	0.91	0.88	0.90
9	0.90	0.81	0.85
Accuracy		0.90	
Macro avg	0.90	0.90	0.90
Weighted avg	0.90	0.90	0.90

TABLE 2. Classification Report For 1 Hidden Layer ANN with ReLU Activation Function and Kaiming Initialization

Class	Precision	Recall	F1-Score
0	0.94	0.99	0.96
1	0.91	1.00	0.95
2	0.94	0.87	0.90
3	0.93	0.87	0.90
4	0.85	0.93	0.88
5	0.87	0.84	0.85
6	0.90	0.93	0.91
7	0.91	0.87	0.89
8	0.88	0.89	0.89
9	0.84	0.79	0.82
Accuracy		0.90	
Macro avg	0.90	0.90	0.90
Weighted avg	0.90	0.90	0.90

APPENDIX I TABLES

TABLE 3. Classification Report For 1 Hidden Layer ANN with ReLU Activation Function and Xavier Initialization

Class	Precision	Recall	F1-Score
0	0.95	1.00	0.97
1	0.91	0.99	0.95
2	0.91	0.87	0.89
3	0.87	0.83	0.85
4	0.87	0.95	0.91
5	0.88	0.82	0.85
6	0.92	0.92	0.92
7	0.89	0.91	0.90
8	0.92	0.87	0.90
9	0.86	0.83	0.84
Accuracy		0.90	
Macro avg	0.90	0.90	0.90
Weighted avg	0.90	0.90	0.90

TABLE 4. Classification Report For 1 Hidden Layer ANN with Sigmoid Activation Function and Random Initialization with Logistic Regression

Class	Precision	Recall	F1-Score
0	0.90	1.00	0.95
1	0.87	0.98	0.92
2	0.89	0.85	0.87
3	0.84	0.83	0.84
4	0.80	0.82	0.81
5	0.84	0.70	0.77
6	0.87	0.89	0.88
7	0.84	0.89	0.86
8	0.83	0.83	0.83
9	0.72	0.64	0.68
Accuracy		0.84	
Macro avg	0.84	0.84	0.84
Weighted avg	0.84	0.84	0.84

TABLE 5. Classification Report For 1 Hidden Layer ANN with Tanh Activation Function and Random Initialization with Logistic Regression

Class	Precision	Recall	F1-Score
0	0.91	1.00	0.95
1	0.98	0.99	0.99
2	0.92	0.87	0.90
3	0.85	0.87	0.86
4	0.87	0.95	0.91
5	0.89	0.78	0.83
6	0.90	0.94	0.92
7	0.91	0.91	0.91
8	0.91	0.89	0.90
9	0.88	0.84	0.86
Accuracy		0.90	
Macro avg	0.90	0.90	0.90
Weighted avg	0.90	0.90	0.90

TABLE 7. Classification Report for 2 Hidden Layer ANN with ReLU as Activation Function on Both Layers

Class	Precision	Recall	F1-Score	Support
0	0.94	1.00	0.97	92
1	0.93	1.00	0.96	105
2	0.90	0.90	0.90	119
3	0.91	0.83	0.87	90
4	0.90	0.94	0.92	95
5	0.86	0.80	0.83	91
6	0.90	0.95	0.93	98
7	0.94	0.93	0.93	100
8	0.92	0.89	0.91	103
9	0.88	0.84	0.86	107
Accuracy	0.91			
Macro Avg	0.91	0.91	0.91	1000
Weighted Avg	0.91	0.91	0.91	1000

TABLE 6. Classification Report For 1 Hidden Layer (with 100 Neurons) ANN with ReLU Activation Function and Random Initialization with Logistic Regression

Class	Precision	Recall	F1-Score
0	0.94	1.00	0.97
1	0.94	1.00	0.97
2	0.94	0.92	0.93
3	0.94	0.90	0.92
4	0.89	0.96	0.92
5	0.91	0.88	0.89
6	0.97	0.95	0.96
7	0.93	0.93	0.93
8	0.96	0.90	0.93
9	0.90	0.88	0.89
Accuracy		0.93	
Macro avg	0.93	0.93	0.93
Weighted avg	0.93	0.93	0.93

TABLE 8. Number of Iterations for Different Configurations of ANN

Accuracy	Number of Iterations						
	Relu	Sigmoid	Tanh	Kiaming	Xavier	100 Neurons	Multiple Layers
10	5	5	5	5	5	5	5
20	10	25	10	5	15	5	30
30	20	35	10	10	20	5	90
40	30	100	20	15	30	10	100
50	40	165	30	20	35	10	115
60	55	245	45	30	40	15	130
70	70	365	75	45	55	20	160
80	115	600	140	90	100	35	245
90	485	NAN	740	795	485	245	750

ACKNOWLEDGMENT

We would like to express our sincere gratitude to Professor Dinesh Kshetri Baniya (DBK sir) for his invaluable guidance and support throughout this lab exercise on Naive Bayes Classification at IOE Thapathali Campus. His insightful feedback and encouragement have greatly enriched our understanding of the subject matter and enhanced the quality of this report. DBK sir has inspired us to undertake deep analysis of the topic, which has helped us gain valuable insights on every topic we explore.

We are also thankful to our classmates and peers for their collaboration and constructive discussions during the lab sessions. Their diverse perspectives and inputs have contributed significantly to the depth and breadth of our learning experience.

Additionally, we would like to acknowledge Department of Electronics and Computer Engineering, IOE, Thapathali Campus for providing the necessary resources and facilities that enabled us to conduct this research effectively.

Last but not least, we extend our appreciation to our families and friends for their unwavering encouragement and understanding during this academic endeavor.

REFERENCES

- [1] Patel, P., Nandu, M., Raut, P. (2018). Initialization of Weights in Neural Networks. International Journal of Scientific Development and Research (IJS DR), 3(11), 73-79.
- [2] Maind, S. B., Wankar, P. (2014). Research Paper on Basic of Artificial Neural Network. International Journal on Recent and Innovation Trends in Computing and Communication, 2(1), 96-100.
- [3] Sharma, S., Sharma, S., Athaiya, A. (2020). Activation Functions in Neural Networks. International Journal of Engineering Applied Sciences and Technology, 4(12), 310-316.
- [4] Alexandros Vasileiadis, Eirini Alexandrou, Lydia Paschalidou, Maria Chrysanthou, Maria Hadjichristoforou. "Artificial Neural Network and Its Applications."
- [5] P. N. Tan, M. Steinbach, A. Karpatne, and V. Kumar, "Introduction to Data Mining," India: Pearson, 2009.



RIJAN POKHREL is currently pursuing a Bachelor's Degree in Electronics and Information Engineering at IOE Thapathali Campus, as of 2024. His passion lies in leveraging AI to revolutionize industries and solve complex problems. He is a motivated learner, constantly staying updated with the latest advancements in AI. Currently exploring quantum machine learning and proficient in app development using Flutter and backend development with Firebase.

Passionate about mathematics, statistics, and machine learning, R. Pokhrel possesses a keen interest in discovering hidden insights from data. He aspires to bridge the financial sector with automated technology through artificial intelligence. Additionally, he is a strong advocate for free speech and is deeply interested in politics and the stock market.



SABIN ACHARYA is an undergraduate student studying Electronics, Communication and Information Engineering in Institute of Engineering, Thapathali Campus, Tribhuvan University graduating in April 2025.

He is a Tech-enthusiast with keen interests in circuits and electronics along with Machine Learning and Software Development. He has skills like Android Application Development with Flutter, Backend with Node.js and Machine Learning Skills with Python. His recent projects include "Virtual Simulation of Hand Gestures Using Hall Effect Sensors". Apart from academics, he is keenly interested in portrait sketching and music.

Mr. Acharya was the recipient of Semester Topper Award from Thapathali Campus in the respective fields for 4 times till March 2024.

...